TABLE OF CONTENTS

CHAPTER NO:	TITLE	PAGE NO:
	ABSTRACT	i
	LIST OF TABLES	ii
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	**INTRODUCTION**	1
	1.1 OVERVIEW OF THE PROJECT	2
	1.2 LITERATURE SURVEY	3
	1.2.1 Machine Learning-Based Water Quality Prediction Systems.	3
	1.2.2 Water Quality Index-Based Water Assessment Systems.	3
	1.2.3 AI-Based Multi-Source Water Quality Prediction.	4
	1.2.4 Machine Learning Techniques for Water Quality Monitoring.	4
	1.2.5 Intelligent Water Quality Assessment and Monitoring Systems.	4
	1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES	5
	1.3.1 Objectives of the Proposed system	5
	1.3.2 Scope of the Proposed system	6
2	**REQUIREMENTS SPECIFICATIONS**	8
	2.1 OVERALL DESCRIPTION	9
	2.1.1 Functional Requirements	10-11
	2.2 PRODUCT PERSPECTIVE	11-12
	2.3 PRODUCT FUNCTIONS	12
	2.3.1 Water Source Selection and Parameter Configuration.	12
	2.3.2 AI-Based Water Quality Prediction.	13
	2.3.3 Water Quality Parameter Submission.	13
	2.3.4 Water Quality Analysis.	13
	2.3.5 Safe and Unsafe Water Classification.	14

	2.3.6 Water Quality Reporting and Records	14
	2.3.7 Appointment and Inspection Management.	14
	2.3.8 User Account and Water Quality History Management	14
	2.3.9 Admin and Inspector Management	14
	2.4 USER CHARACTERISTICS	15
	2.4.1 Admin	16
	2.4.2 User	16
	2.4.3 Inspector	17
	2.5 OPERATING ENVIRONMENT CONSTRAINTS	17
	2.5.1 Software Specification	17
	2.5.2 Hardware Specification	18
	2.6 SPECIFIC REQUIREMENTS	18
	2.6.1 User Requirements	18
	2.6.2 Admin Requirements	19
	2.6.3 Functional Requirements	19
	2.6.4 Non- Functional Requirements	20
	2.6.5 Security Requirements	21
	2.7 EXTERNAL INTERFACE REQUIREMENTS	21-22
	2.8 SYSTEM FEATURES	22-23
3	**SYSTEM DESIGN AND TEST PLAN**	24
	3.1 DATA FLOW DIAGRAM (DFD)	25
	3.1.1 Level 0 DFD	25-26
	3.1.2 Level 1 DFD	26-28
	3.2 USE CASE DIAGRAM	28
	3.2.1 Admin Use Case Diagram	29
	3.2.2 User Use Case Diagram	29-30
	3.2.3 Patient Use Case Diagram	30-31
	3.3 TABLE DESIGNS	33
	3.3.1 User Table	31
	3.3.2 Inspector Table	32
	3.3.3 Prediction Table	33
	3.3.4 Payments Table	33-34

	3.3. inspection_reports Table	34
	3.3.6 field_inspection Table	34-35
	3.3.7 appointments Table	35-36
	3.3.8 feedbacks Table	36
	3.4 PERFORMANCE REQUIREMENTS	36-37
	3.5 SOFTWARE QUALITY ATTRIBUTES	37-38
	3.6 DETAILED DESIGN	38
	3.6.1 System Architecture	38
	3.6.2 User Registration and Authentication	39
	3.6.3 User Profile and Water Quality Management	39
	3.6.4 AI-Based Water Quality Prediction	39-40
	3.6.5 Water Quality Data Processing and Evaluation.	40
	3.6.6 Reporting, Inspection and Monitoring.	40
	3.6.7 Security Measures	40
	3.6.8 Performance Optimization	40
	3.6.9 Testing and Deployment	41
	3.7 TEST CASES	41
	3.7.1 Test Cases for Admin	41-42
	3.7.2 Test Cases for User	42-43
	3.7.3 Test Cases for Inspector	43-44
4	**IMPLEMENTATION AND RESULTS**	46
	4.1 IMPLEMENTATION	46
	4.1.1 Backend Development	46-47
	4.1.2 Frontend Development	47-48
	4.1.3 Integration and Testing	48-49
	4.2 IMPLEMENTATION RESULTS	49-50
5	**CONCLUSION AND FUTURE WORKS**	51
	5.1 CONCLUSION	51-52
	5.2 FUTURE WORKS	52
	5.2.1 Future Implementation Plan	52-53
	REFERENCES	54
	APPENDIXES	55

	Appendix A- Screenshots	55-57
	Appendix B- Sample Code	57-86

ABSTRACT

Aquaguard is an AI-Based Multi-Source Water Quality Prediction and Monitoring System developed to assess and predict the quality of water obtained from different sources such as well water, river water, tap water, lake water, and industrial water. The main objective of the system is to provide an efficient and user-friendly platform for evaluating water quality using various water quality parameters and predicting whether the water is safe or unsafe. The system uses a Water Quality Index (WQI) dataset containing different parameters related to water quality and applies machine learning techniques to develop a prediction model. The collected or entered water quality parameter values are processed by the system and provided to the trained machine learning model, which predicts the quality status of the water. The system allows users to enter the required water quality parameters and view the corresponding water quality result in a simple and understandable manner. When the predicted water quality is unsafe, the system provides an option for further inspection through an inspector, allowing the water quality issue to be examined based on the inspection process. The system also maintains relevant information related to users, predictions, reports, appointments, and inspections, helping to organize the water quality assessment process. Aquaguard combines water quality analysis, machine learning-based prediction, and inspection management into a single platform, making the process of water quality assessment more systematic and accessible. The machine learning model is trained using available water quality data to identify patterns in the parameters and generate water quality predictions. By providing a digital approach to water quality prediction and monitoring, Aquaguard helps users understand the condition of their water and supports the identification of unsafe water through an organized assessment and inspection workflow. Overall, the system is designed to provide a practical solution for multi-source water quality prediction and monitoring by integrating water quality parameters, WQI-based assessment, machine learning prediction, and inspection-related processes within one system.

.

LIST OF TABLES

TABLE NO:	TITLE	PAGE NO:
3.1	users	33-34
3.2	Inspectors	34-35
3.3	Prediction	35
3.4	Payments	36
3.5	inspection_reports	36-37
3.6	field_inspection	37-38
3.7	appointments	38
3.8	feedbacks	39

LIST OF FIGURES

FIGURE NO:	TITLE	PAGE NO:
3.1	DFD Level 0	28
3.2	DFD Level 1	29
3.3	Admin Use Case Diagram	31
3.4	User Use Case Diagram	32
3.5	Inspector Use Case Diagram	32
A.1	Home Page	63
A.2	Login Page	63
A.3	Admin Dashboard	64
A.4	User Dashboard	64
A.5	Inspector Dashboard	65

LIST OF ABBREVIATIONS

LIST	ABBREVIATIONS	MEANING
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSS	Cascading Style Sheets
4	DFD	Data Flow Diagram
6	HTML	Hypertext Markup Language
7	HTTP	Hypertext Transfer Protocol
8	ML	Machine Learning
9	RF	Random Forest
10	SVM	Support Vector Machine

CHAPTER 1
INTRODUCTION

In today's rapidly developing technological environment, the need for efficient and intelligent water quality assessment systems has increased significantly due to the importance of safe water for daily use and the difficulties involved in manually assessing water quality. Traditional methods of water quality assessment can require laboratory analysis and manual evaluation of different water quality parameters, which may take time and make it difficult for users to obtain an immediate understanding of their water condition. To address these issues, the "Aquaguard" has been developed as an AI-Based Multi-Source Water Quality Prediction and Monitoring System that uses machine learning techniques to assess and predict water quality from different sources.

The proposed system is designed to provide a platform for analysing water quality from multiple sources such as well water, river water, tap water, lake water, and industrial water. The system uses various water quality parameters to evaluate the condition of the water and predict whether it is safe or unsafe. A Water Quality Index (WQI) dataset is used for developing the machine learning model, and the available water quality data is processed to identify patterns and relationships between different parameters. Based on the entered water quality values, the trained machine learning model predicts the water quality and provides the corresponding result to the user.

In addition to water quality prediction, the system includes features for managing the water quality assessment and inspection process. Users can enter their water quality parameters and obtain the predicted result through the system. When the water is identified as unsafe, the system provides an option for further inspection through an inspector. The inspection process allows the water condition to be examined based on the provided water quality information and inspection details. The system also maintains relevant records such as user information, predictions, reports, appointments, and inspection-related details, helping to organize the overall water quality assessment process.

The system is developed using Python, Flask, HTML, CSS, JavaScript, and MySQL, with machine learning techniques used for water quality prediction. The machine learning model is trained using the available WQI dataset to generate water quality predictions based on the input parameters. By combining water quality parameter analysis, Water Quality Index assessment, machine learning-based prediction, and inspection management, Aquaguard

provides a systematic platform for assessing and monitoring water quality from different sources. The project aims to make water quality prediction more accessible and understandable to users while supporting the identification and further inspection of unsafe water through a single integrated system.

.

1.1 OVERVIEW OF THE PROJECT

The "Aquaguard" is an AI-Based Multi-Source Water Quality Prediction and Monitoring System designed to assess water quality, predict whether water is safe or unsafe, and support the inspection and monitoring of water quality through an integrated web-based platform. The project combines machine learning-based water quality prediction, Water Quality Index analysis, user management, and inspection-related modules into a single system capable of handling water quality assessment from different sources.

The system enables users to register and enter the required water quality parameters for assessment. The platform supports water quality prediction for different sources such as well water, river water, tap water, lake water, and industrial water. The entered water quality parameters are processed and analysed using a trained machine learning model to predict the quality status of the water. The system uses a Water Quality Index (WQI) dataset containing different water quality parameters for developing the prediction model and assessing the condition of water.

The water quality assessment component of the system allows users to submit water quality parameter values and obtain the corresponding prediction result. Based on the analysed parameters, the system identifies whether the water is safe or unsafe. The system also maintains the relevant prediction information so that the water quality assessment can be managed in an organized manner. When the predicted result indicates that the water is unsafe, the user can proceed with the inspection process through an inspector.

The inspection management component allows administrators to manage inspectors and assign them for water quality inspection. Inspectors can review the assigned inspection details and perform the required inspection based on the available water quality information. The system also supports appointment management and maintains relevant records related to users, predictions, reports, appointments, and inspections, helping to organize the overall water quality assessment and monitoring process.

The system is developed using Python, Flask, HTML, CSS, JavaScript, and MySQL,

with machine learning techniques integrated for water quality prediction. The trained machine learning model analyses the water quality parameters and generates predictions based on the available water quality data. The platform provides a user-friendly interface through which users, administrators, and inspectors can perform their respective activities related to water quality assessment and inspection.

Overall, Aquaguard serves as an intelligent water quality prediction and monitoring platform that combines machine learning-based prediction, Water Quality Index analysis, multi-source water assessment, and inspection management. The system aims to provide a systematic and accessible approach to understanding water quality and identifying unsafe water through automated prediction and an organized inspection process.

1.2 LITERATURE SURVEY

1.2.1 Machine Learning-Based Water Quality Prediction Systems.

The study **"Machine Learning-Based Water Quality Prediction Systems" (2025)** explores the application of machine learning techniques for predicting water quality using different physical and chemical water quality parameters. The system analyses parameters related to water quality and uses trained machine learning models to identify the overall condition of water. Such systems help reduce the dependency on completely manual water quality assessment methods by providing automated predictions based on previously available water quality data. The study demonstrates the usefulness of machine learning in identifying patterns within water quality datasets and predicting water quality conditions. It highlights the potential of AI and machine learning techniques in developing efficient water quality assessment systems.

1.2.2 Water Quality Index-Based Water Assessment Systems.

The study **"Water Quality Index-Based Water Assessment Systems" (2024)** investigates the use of the Water Quality Index (WQI) for evaluating the overall quality of water. WQI provides a systematic method of representing water quality by considering multiple water quality parameters and combining them into an index value. The study examines the importance of parameters such as pH, dissolved oxygen, conductivity, turbidity, and other

relevant characteristics in determining water quality. The research shows that WQI-based assessment provides a simple and understandable method for representing complex water quality information and can be effectively used for water quality classification and monitoring.

1.2.3 AI-Based Multi-Source Water Quality Prediction.

The study **"AI-Based Multi-Source Water Quality Prediction" (2025)** examines the application of artificial intelligence and machine learning for assessing water obtained from different sources. Water quality can vary depending on its source and environmental conditions, making it important to analyse water parameters before determining its quality. The study focuses on using machine learning models to analyse water quality data from different sources and generate predictions regarding the condition of water. The research demonstrates that AI-based prediction systems can identify relationships between different water quality parameters and support automated water quality assessment. Such approaches can be applied to sources including surface water, groundwater, and other available water sources.

1.2.4 Machine Learning Techniques for Water Quality Monitoring.

The study **"Machine Learning Techniques for Water Quality Monitoring" (2025)** investigates the role of machine learning in analysing and monitoring water quality data. Traditional water quality monitoring involves collecting samples and analysing several parameters, which can require considerable time and resources. The study explores how machine learning models can process historical and measured water quality data to identify patterns and provide predictions about water conditions. The research highlights the usefulness of machine learning algorithms such as Random Forest and other predictive techniques in water quality analysis. The study concludes that integrating machine learning with water quality monitoring can provide an efficient approach for analysing large amounts of water quality information..

1.2.5 Intelligent Water Quality Assessment and Monitoring Systems.

The study **"Intelligent Water Quality Assessment and Monitoring Systems" (2026)** examines the development of intelligent systems for analysing, assessing, and monitoring water quality using modern computational techniques. The research focuses on the importance of

analysing multiple water quality parameters to determine whether water is suitable or unsuitable based on its measured characteristics. By integrating water quality data analysis with machine learning-based prediction, intelligent systems can provide faster and more organized water quality assessment. The study highlights the importance of automated prediction and monitoring in improving the accessibility and efficiency of water quality evaluation. It demonstrates how intelligent water quality systems can support the identification of unsafe water conditions and provide a systematic approach to water quality assessment.

1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES

1.3.1 Objectives of the Proposed System

The primary objective of the "Aquaguard" project is to develop an AI-Based Multi-Source Water Quality Prediction and Monitoring System that assists users in assessing and understanding the quality of water through machine learning and water quality analysis techniques. The system aims to provide a user-friendly platform where users can enter different water quality parameters and obtain an automated prediction regarding whether the water is safe or unsafe. By applying machine learning techniques to water quality data, the system seeks to reduce the complexity involved in analysing multiple water quality parameters and provide a clear water quality assessment.

Furthermore, the system is designed to assess water from different sources such as well water, river water, tap water, lake water, and industrial water. The proposed system uses a Water Quality Index (WQI) dataset containing different water quality parameters for developing and training the machine learning model. The system analyses the entered parameter values and uses the trained model to predict the quality condition of the water. This helps users obtain a better understanding of their water quality based on the available parameters and prediction results.

In addition, the system aims to support the inspection process when unsafe water is identified. Users can proceed with an inspection through an inspector, while administrators can manage inspectors and assign them for the required inspection. Inspectors can view the assigned inspection information and carry out the inspection based on the available water quality details. The system also maintains relevant information related to users, predictions, reports, appointments, and inspections, thereby helping to organize the overall water quality assessment process. Overall, the proposed system aims to provide a systematic approach to

water quality prediction and monitoring by combining water quality parameter analysis, WQI-based assessment, machine learning prediction, and inspection management within a single platform.

1.3.2 Scope of the Proposed System

The scope of the "Aquaguard" project includes the design and development of an AI-based web platform for predicting and monitoring water quality from multiple sources. The system is intended to provide users with an accessible method for entering water quality parameters, analysing the submitted values, and obtaining a prediction of the water quality condition. The platform focuses on water quality assessment using machine learning techniques and supports the evaluation of water from sources such as well, river, tap, lake, and industrial water.

The platform enables users to provide the required water quality parameters for assessment and receive a prediction indicating whether the water is safe or unsafe. The system uses the available WQI dataset for machine learning-based water quality prediction and processes the input parameters through the trained model. The water quality assessment allows users to understand the condition of their water based on the parameters provided. The system also maintains prediction information and relevant water quality records as part of the overall assessment process.

The proposed system also includes an inspection management component for unsafe water conditions. When water is predicted to be unsafe, an inspection can be arranged through an inspector. Administrators can register and manage inspectors and assign them to users for inspection. Inspectors can access the assigned inspection details and perform the required field inspection based on the available information. The system also includes appointment and report-related functionalities that support the organization of the water quality assessment and inspection process.

Additionally, the system provides administrative functionality for managing users, inspectors, assignments, appointments, predictions, reports, and inspection-related information. The current implementation primarily focuses on multi-source water quality prediction, Water Quality Index-based assessment, machine learning analysis, and inspection management. The system is developed using Python, Flask, HTML, CSS, JavaScript, and MySQL, with machine learning techniques used for water quality prediction. Overall, Aquaguard aims to provide a systematic, accessible, and intelligent platform for assessing

water quality and identifying unsafe water through automated prediction and an organized inspection process.

CHAPTER 2
REQUIREMENTS SPECIFICATIONS

The requirement specification for the "Aquaguard" project outlines the development of an AI-Based Multi-Source Water Quality Prediction and Monitoring System using modern technologies such as Python, Flask framework, machine learning techniques, HTML, CSS, JavaScript, and MySQL. The system is designed to assist users, administrators, and inspectors by providing water quality prediction, water quality assessment, appointment management, and inspection-related services through a centralized web-based platform. The system incorporates machine learning techniques for analysing water quality parameters and predicting the quality condition of water based on the available Water Quality Index (WQI) dataset.

Key functionalities of the system include user authentication and profile management, water quality parameter entry, machine learning-based water quality prediction, Water Quality Index-based assessment, appointment management, inspector assignment, inspection management, and report-related functionalities. The platform enables users to enter the required water quality parameters for different water sources such as well water, river water, tap water, lake water, and industrial water. The entered values are processed by the system and analysed using the trained machine learning model to generate a prediction indicating whether the water is safe or unsafe.

The platform also supports the inspection process for water identified as unsafe. When an unsafe water condition is predicted, the user can proceed with an inspection through an inspector. The administrator is responsible for managing inspectors and assigning inspectors to the required users. Inspectors can access their assigned inspection details and carry out the inspection based on the available water quality information. The system also supports appointment management and maintains relevant information related to users, predictions, reports, appointments, and inspections to ensure that the water quality assessment process is properly organized.

Additionally, the system provides functionalities for managing water quality prediction and inspection information through dedicated interfaces for the respective users. The machine learning component processes the water quality parameters and uses the trained model to generate the prediction result. The system maintains the relevant records in the MySQL database, allowing user, prediction, appointment, report, and inspection information to be managed within the platform. The web interface is developed using HTML, CSS, and

JavaScript, while Python and Flask are used for the application backend and integration of the machine learning model.

Overall, the Aquaguard aims to provide a systematic, user-friendly, and intelligent water quality assessment solution that combines multi-source water quality analysis, Water Quality Index assessment, machine learning-based prediction, and inspection management within a single web-based platform. The system helps users understand the quality condition of their water and supports the identification and further inspection of unsafe water through an organized digital process.

2.1 OVERALL DESCRIPTION

The Aquaguard project is an AI-Based Multi-Source Water Quality Prediction and Monitoring System developed using modern technologies such as Python and the Flask framework, integrated with machine learning techniques for water quality prediction and analysis. The platform is designed to provide an intelligent and interactive water quality assessment environment that assists users, administrators, and inspectors in assessing and monitoring water quality efficiently. It provides a backend system that manages water quality prediction, water quality assessment, appointment scheduling, inspector assignment, inspection details, and reports while ensuring proper data processing and management. The system addresses the growing need for an efficient water quality assessment platform where users can enter different water quality parameters and obtain an automated prediction of the water condition. The platform supports water quality assessment from multiple sources such as well water, river water, tap water, lake water, and industrial water. By incorporating machine learning-based water quality prediction and Water Quality Index (WQI) analysis, the platform helps users identify whether the water is safe or unsafe. When unsafe water is identified, the system supports further inspection through an inspector, allowing the water quality assessment process to continue in an organized manner. Overall, Aquaguard provides a user-friendly platform that combines water quality parameter analysis, machine learning-based prediction, WQI-based assessment, and inspection management for effective water quality monitoring.

.

2.1.1 Functional Requirements:

The functional requirements for the Aquaguard project are designed to provide a comprehensive and intelligent water quality assessment and monitoring solution for users, administrators, and inspectors. The platform enables users to register, log in, and manage their profiles, providing personalized access to the water quality assessment functionalities. Users can select the required water source and enter the relevant water quality parameters, after which the system analyses the submitted values and generates a water quality prediction indicating whether the water is safe or unsafe.

To ensure effective water quality assessment, the system integrates machine learning techniques for analysing different water quality parameters. The prediction model is developed using the available Water Quality Index (WQI) dataset and processes the entered water quality values to generate the water quality result. The system supports water quality assessment for different sources such as well water, river water, tap water, lake water, and industrial water. The machine learning model analyses the input parameters and provides the corresponding prediction, helping users understand the condition of their water.

The platform provides appointment and inspection management functionalities for cases where the water is identified as unsafe. Users can book an appointment with an inspector when further inspection is required. Administrators can manage inspectors and assign them to the required users for inspection. Inspectors can access their assigned inspection details and carry out the inspection based on the available water quality information. The system also maintains the relevant inspection details and allows the inspection process to be managed through the platform.

The system includes reporting and record management functionalities for maintaining water quality predictions, appointments, inspection information, and reports. The relevant information is stored in the MySQL database, allowing the system to maintain and retrieve the required records. Users can view their water quality prediction results, while administrators and inspectors can access the information required for their respective activities within the system.

To provide controlled access to the different functionalities, the platform includes authentication and role-based access for users, administrators, and inspectors. The admin module provides functionalities for managing users and inspectors, assigning inspectors, and monitoring relevant water quality assessment and inspection activities. Inspectors can view assigned inspection information and update the required inspection details, while users can enter water quality parameters, view prediction results, and access inspection-related services

when required.

The backend system, developed using Python and Flask, manages communication between the web interface, machine learning model, and MySQL database to process water quality inputs and maintain system information. The frontend is developed using HTML, CSS, and JavaScript, providing interfaces for the different users of the system. Overall, the functional requirements aim to deliver a user-friendly and intelligent water quality platform that integrates water quality parameter analysis, WQI-based assessment, machine learning prediction, appointment management, inspector assignment, inspection management, and report management to support systematic water quality assessment and monitoring.

2.2 PRODUCT PERSPECTIVE

The **Aquaguard** is positioned as an AI-Based Multi-Source Water Quality Prediction and Monitoring System designed to assess and monitor water quality through the integration of machine learning and modern web technologies. The system is developed using **Python and the Flask framework**, providing a structured web-based environment for water quality assessment, prediction, appointment management, inspector assignment, and inspection-related activities. The platform is designed to support users, administrators, and inspectors within a centralized system for managing the water quality assessment process.

At the core of the system, the **MySQL database** is utilized to manage user information, water quality prediction records, appointment details, reports, and inspection-related information. The database provides structured storage and retrieval of the information required for the different activities of the system. The Flask framework provides the backend infrastructure for handling authentication, application routing, user requests, water quality prediction processing, database communication, and interaction between the machine learning model and the frontend interfaces.

The platform integrates **machine learning techniques** to provide intelligent water quality prediction functionality. The machine learning model is developed using the available **Water Quality Index (WQI) dataset** and is used to analyse different water quality parameters. Users can enter the required parameters for water sources such as **well water, river water, tap water, lake water, and industrial water**, and the system processes these values through the trained model to predict whether the water is **safe or unsafe**. The water quality assessment

component provides users with a clear prediction result based on the submitted parameters.

From an interaction perspective, the system provides a user-friendly interface where users can register, log in, enter water quality parameters, view prediction results, and access inspection-related services when required. When the water is predicted to be unsafe, users can proceed with an appointment for inspection. Administrators are provided with functionalities for managing users and inspectors, assigning inspectors to users, and monitoring relevant appointment and inspection information. Inspectors can access their assigned inspection details and perform the required inspection based on the available water quality information.

Functionally, the system supports **user authentication, water quality parameter entry, multi-source water quality prediction, WQI-based assessment, appointment management, inspector assignment, inspection management, and report management**. The system also maintains relevant records related to water quality predictions, users, appointments, inspections, and reports through the MySQL database. The backend manages communication between the frontend, database, and machine learning model to ensure proper processing of water quality inputs and system operations.

In terms of security and reliability, the platform provides authentication and controlled access according to the roles of users, administrators, and inspectors. The system ensures that the respective functionalities and information are accessible according to the activities performed by each role. The platform is designed to provide an organized and reliable environment for managing water quality assessment and inspection information. Overall, **Aquaguard** serves as an intelligent water quality solution that combines machine learning-based prediction, Water Quality Index assessment, multi-source water analysis, and inspection management to provide a systematic and accessible approach to water quality monitoring.

2.3 PRODUCT FUNCTIONS

2.3.1 Water Source Selection and Parameter Configuration.

The **Water Source Selection and Parameter Configuration** functionality allows users to initiate a water quality assessment by selecting the required water source, such as **well water, river water, tap water, lake water, or industrial water**. Based on the selected source, users can provide the required water quality parameters for analysis. The system prepares the submitted information for processing by the water quality prediction model, ensuring that the

assessment is based on the parameters provided by the user.

2.3.2 AI-Based Water Quality Prediction.

The **AI-Based Water Quality Prediction** feature enables the system to analyse the entered water quality parameters using a trained machine learning model. The model is developed using the available **Water Quality Index (WQI) dataset** and processes the submitted values to generate a water quality prediction. The system determines whether the analysed water is **safe or unsafe**, providing users with an automated assessment of the water condition.

2.3.3 Water Quality Parameter Submission.

The **Water Quality Parameter Submission** functionality allows users to enter the required values of different water quality parameters through structured forms. The submitted parameters are collected by the system and passed to the prediction process for analysis. This functionality provides a systematic method for collecting the water quality information required for machine learning-based assessment.

2.3.4 Water Quality Analysis.

The **Water Quality Reporting and Records** feature maintains information related to water quality predictions and assessment activities. The system stores relevant prediction information and report details in the database, allowing the records to be managed as part of the water quality assessment process. These records support the organized maintenance of water quality information within the system.

2.3.5 Safe and Unsafe Water Classification.

The **Safe and Unsafe Water Classification** functionality presents the prediction generated by the machine learning model in an understandable form. After analysing the submitted water quality parameters, the system classifies the water as either **safe or unsafe**. This allows users to clearly identify the predicted condition of their water and determine whether further inspection may be required.

2.3.6 Water Quality Reporting and Records

The **Water Quality Reporting and Records** feature maintains information related to water quality predictions and assessment activities. The system stores relevant prediction information and report details in the database, allowing the records to be managed as part of the water quality assessment process. These records support the organized maintenance of water quality information within the system.

2.3.7 Appointment and Inspection Management.

The **Appointment and Inspection Management** functionality allows users to proceed with an inspection when the water is identified as unsafe. Users can book an appointment with an inspector, while the administrator can manage the appointment and inspection-related information. This functionality supports the continuation of the water quality assessment process from automated prediction to inspection.

2.3.8 User Account and Water Quality History Management

The **User Account and Water Quality History Management** functionality enables users to register, log in, and manage their accounts within the system. Relevant information related to users, water quality predictions, appointments, reports, and inspections is maintained in the database. This allows the system to organize user-related water quality assessment information and provide access to the required records.

2.3.9 Admin and Inspector Management

The **Admin and Inspector Management** feature provides administrators with functionalities for managing inspectors and assigning them to users for inspection. Administrators can manage relevant user and inspector information and monitor appointment and inspection activities. Inspectors can access their assigned inspection details and perform the required inspection based on the available water quality information. This functionality helps maintain an organized workflow between users, administrators, and inspectors throughout the water quality assessment process.

2.4 USER CHARACTERISTICS

2.4.1 Admin

Admins in the **Aquaguard** system are responsible for managing and monitoring the overall functioning of the water quality prediction and inspection platform. They have access to administrative functionalities that allow them to manage users and inspectors, assign inspectors to users, and monitor appointments and inspection activities. Admins can view relevant water quality prediction and inspection information and ensure that the water quality assessment workflow is properly managed. They are also responsible for maintaining the necessary user, inspector, appointment, report, and inspection information within the system. Their role is important in coordinating the activities between users and inspectors and maintaining the overall operational efficiency of the Aquaguard system.

2.4.2 User

Users are the primary users of the **Aquaguard** system who utilize the platform to assess and monitor the quality of their water. Users can register, log in, and manage their accounts before accessing the water quality assessment functionalities. They can select the required water source and enter the relevant water quality parameters to receive a machine learning-based prediction indicating whether the water is safe or unsafe. Users can view their water quality prediction results and, when the water is identified as unsafe, book an appointment with an inspector for further inspection. They can also access relevant prediction, appointment, report, and inspection information associated with their water quality assessment.

2.4.3 Inspector

Inspectors in the **Aquaguard** system are responsible for carrying out inspections related to water quality when an inspection is assigned to them. Inspectors can access the system and view the inspection details assigned by the administrator. They can review the available water

quality information and perform the required inspection based on the assigned request. Inspectors can also update the relevant inspection details through the system, allowing the inspection information to be maintained along with the user's water quality assessment records. Their role is essential in providing further assessment of water when the machine learning prediction identifies the water as unsafe.

2.5 OPERATING ENVIRONMENT CONSTRAINTS

The **Aquaguard** system integrates web technologies, machine learning, and database management to provide water quality prediction and monitoring. The system uses **HTML, CSS, and JavaScript** for the frontend, **Python and Flask** for the backend, machine learning libraries for water quality prediction, and **MySQL** for data storage and management. The system requires a suitable Python environment with the necessary libraries and a trained machine learning model based on the **WQI dataset**. A stable database connection and modern web browser are required for proper system operation and communication between the frontend, backend, machine learning model, and database

2.5.1 Software Specifications

Software requirements are the technical specifications necessary for the successful development, deployment, and execution of the **Aquaguard – AI-Based Multi-Source Water Quality Prediction and Monitoring System**. These requirements ensure proper water quality prediction, data processing, database management, and smooth operation of the web application.

Operating System	:	Windows 10/11, Ubuntu 20.04+, or macOS
Web Browser	:	Any Standard Web Browser
Frontend	:	HTML, CSS, JavaScript
Backend	:	Python Flask
Database	:	MySQL
Programming Language	:	Python 3.12+
Machine Learning Libraries	:	Scikit-learn, Pandas, NumPy

AI Model Integration	:	Joblib
Development Environment	:	VS Code
Version Control	:	Git & GitHub

2.5.2 Hardware Specifications

The hardware requirements define the minimum system specifications necessary for the **Aquaguard AI system** to function efficiently. These specifications ensure smooth execution of machine learning models, water quality data processing, prediction generation, inspection management, and other system operations.

Processor	:	Intel i5 / AMD Ryzen 5 or higher
RAM	:	8GB Minimum
Storage	:	1 GB or more of free disk space
Display	:	Standard Monitor with 1366×768 Resolution or Higher
Peripherals		Keyboard and Mouse

2.6 SPECIFIC REQUIREMENTS

The specific requirements for the **Aquaguard AI system** outline the detailed functionalities and operational constraints necessary for developing an intelligent, secure, and efficient water quality monitoring and management platform. These requirements are designed to ensure that the system provides accurate water quality prediction, effective water quality assessment, inspection management, appointment scheduling, and appropriate recommendations for maintaining safe water.

The platform integrates **artificial intelligence and machine learning technologies** to analyze water quality parameters, calculate water quality indicators, predict whether water is safe or unsafe for use, and assist users in understanding potential water-related risks. It also supports multiple water sources, including **well, river, tap, lake, and industrial water**, while providing appropriate interaction between users, inspectors, and administrators.

The system provides functionalities such as **user registration and authentication, water quality parameter entry, AI-based prediction, water quality report generation,**

inspector assignment, online and field inspection management, appointment scheduling, and administrative monitoring. The structured requirement analysis ensures that the platform remains **scalable, reliable, secure, user-friendly, and capable of efficiently supporting water quality assessment and monitoring operations**.

2.6.1 User Requirements:

The user requirements of the **Aquaguard AI system** focus on providing secure, user-friendly, and efficient water quality assessment and monitoring services. The system must allow users to register and authenticate securely using valid credentials such as email and password while protecting user information and maintaining secure sessions. A password recovery mechanism should also be provided to allow users to regain access to their accounts when they forget their login credentials.

Users should be able to **manage their profiles, select the water source, enter water quality parameters, upload laboratory reports where applicable, and obtain AI-based water quality predictions**. The system should display the predicted water quality status, relevant water quality information, and recommendations based on the analysis. Users should also be able to view their prediction history, generate or access water quality reports, and request an inspection when required.

Users should be able to **book appointments with available inspectors** for further water quality assessment. The system should provide inspection-related information and allow users to track the status of their inspection requests.

Inspectors should be able to view assigned inspection requests, access the user's location through a map, review submitted water quality information, conduct online or field inspections, enter observations and measured parameters, and submit inspection reports.

Administrators should be able to manage users and inspectors, assign inspectors to inspection requests, monitor appointments and inspections, review system activities, and manage the overall operation of the platform through a centralized dashboard.

.

2.6.2 Admin Requirements:

The admin requirements focus on providing complete control and monitoring capabilities for effective management of the **Aquaguard AI system**. Administrators must be able to manage user and inspector accounts, monitor water quality prediction activities, oversee

inspection requests, and manage appointments through a centralized administrative dashboard.

The admin module should provide **analytical dashboards** displaying relevant system statistics, including the number of registered users, inspectors, water quality predictions, safe and unsafe water results, appointments, and completed inspections. Administrators should be able to register and manage inspectors, assign inspectors to user inspection requests, and monitor the status of assigned inspections.

Administrators must also be capable of **activating, deactivating, updating, or removing user and inspector accounts** whenever necessary. They should be able to review submitted water quality information and inspection activities to ensure proper functioning of the system. The admin should also monitor AI-generated water quality predictions and maintain the reliability and consistency of the prediction system.

In addition, administrators should have access to **reporting and monitoring tools** for generating system-wide water quality reports, inspection summaries, prediction statistics, appointment details, and operational reports. These functionalities enable administrators to efficiently manage the platform, monitor system activities, and maintain the overall reliability and security of the Aquaguard AI system.

2.6.3 Functional Requirements:

The functional requirements of the **Aquaguard AI system** define the major operations and intelligent functionalities supported by the platform. The system must provide secure authentication and role-based access control to ensure that only authorized users, inspectors, and administrators can access their respective features and functionalities.

The system should support an **AI-based water quality prediction module** capable of analyzing various water quality parameters using machine learning algorithms. Users should be able to select different water sources, such as **well water, river water, tap water, lake water, and industrial water**, and enter the required water quality parameters for analysis. The system must process the submitted data and generate predictions indicating the water quality status, such as whether the water is **safe or unsafe**, along with relevant recommendations and precautionary measures.

The platform should provide functionalities for **water quality report generation, prediction history management, laboratory report upload and parameter extraction where applicable, appointment booking, inspector assignment, and inspection management**. Users should be able to request an inspection when necessary, while inspectors

must be able to review assigned requests, access location information, conduct online or field inspections, record observations, enter measured parameters, and submit inspection reports.

Administrators should be able to manage users and inspectors, assign inspectors to inspection requests, monitor predictions and appointments, and oversee overall system activities through a centralized dashboard. The system should also provide notifications or status updates related to appointments, inspector assignments, and inspection progress.

Efficient communication between the **frontend interface, backend server, machine learning models, database, and map/location services** must be ensured to support smooth water quality prediction, inspection management, and overall system operation.

2.6.4 Non- Functional Requirements:

The non-functional requirements of the **Aquaguard AI system** focus on maintaining high levels of performance, reliability, security, scalability, and usability. The system should be capable of supporting multiple users and processing increasing volumes of water quality data without significantly affecting its performance.

The platform should provide a **responsive and user-friendly interface** that is compatible with desktops, tablets, and smartphones. The system should ensure easy navigation and accessibility for users, inspectors, and administrators. Security should be maintained through secure authentication, role-based access control, and appropriate protection of user and water quality data.

The system architecture should be **scalable** to accommodate an increasing number of users, water quality records, prediction requests, appointments, and inspection activities. High availability and reliability should be maintained through appropriate database management, regular backups, error handling, and recovery mechanisms.

Furthermore, the system should maintain **low response times** for water quality prediction, report generation, appointment booking, inspector assignment, and inspection-related operations. The machine learning model should provide predictions efficiently while maintaining acceptable accuracy. Overall, the system should provide a **reliable, secure, efficient, and user-friendly environment** for water quality assessment and monitoring.

2.6.5 Security Requirements:

The security requirements of the **Aquaguard AI system** focus on protecting user

information, water quality data, inspection records, and system activities from unauthorized access. The system must use **secure authentication and authorization mechanisms** to ensure that only registered and authorized users can access the platform.

User passwords should be stored using secure **password hashing techniques** and should never be stored in plain text. The system should implement **role-based access control (RBAC)** to provide appropriate access to users, inspectors, and administrators according to their roles.

All sensitive data transmitted between the frontend and backend should be protected using **HTTPS**. Important operations such as profile updates, water quality data submission, appointment booking, inspector assignment, inspection updates, and report access should require proper authentication and authorization.

The system should also implement input validation, secure session management, and appropriate database protection to prevent unauthorized access and common security threats. Regular backups should be maintained to avoid data loss.

Overall, the Aquaguard AI system should ensure the **confidentiality, integrity, and availability** of user and water quality information while providing a secure and trustworthy environment for water quality monitoring and assessment.

2.7 EXTERNAL INTERFACE REQUIREMENTS

AquaGuard AI is a web-based application that provides an interactive interface for Users, Inspectors, and Admins. The system can be accessed through a computer, laptop, tablet, or smartphone using a web browser and an internet connection. The application is developed using HTML, CSS, JavaScript, Python, and Flask, with MySQL used for storing and managing system data. Scikit-learn is used for machine learning-based water quality prediction. The system communicates with the database through the backend and uses HTTP/HTTPS for communication between the client and server. External services such as Leaflet/OpenStreetMap are used for map and location-based features, while Chart.js is used for graphical visualization of water quality results.

2.8 SYSTEM FEATURES

The **Aquaguard** system provides a set of features designed to assess water quality, predict whether water is safe or unsafe, and support further inspection when required. The system combines machine learning, water quality analysis, web technologies, and database management to provide an organized water quality monitoring platform. The major system features are as follows.

Multi-Source Water Quality Assessment: The system allows users to select different water sources such as **well water, river water, tap water, lake water, and industrial water** and provide the required water quality parameters for assessment.

AI-Based Water Quality Prediction: The system uses a trained machine learning model based on water quality data to analyze the submitted parameters and generate a water quality prediction. The prediction helps determine whether the provided water is **safe or unsafe** for further consideration.

Water Quality Index Analysis: The system uses the **Water Quality Index (WQI)** based water quality dataset and analysis approach to support the assessment of water quality. The submitted water parameters are processed by the system to provide the corresponding water quality result.

Safe and Unsafe Water Classification: Based on the prediction and analysis, the system classifies the water as **safe or unsafe**. This classification helps users understand the quality condition of the tested water and decide whether further inspection is required.

Inspector Appointment and Assignment: When the water is identified as unsafe, the user can book an appointment for further inspection. The administrator can manage inspectors and assign an appropriate inspector to the inspection request.

Field Inspection Management: Inspectors can access their assigned inspection details and perform the required inspection. The inspector can update the inspection information based on the observations and findings during the inspection process.

User, Admin and Inspector Management: The system provides separate functionalities for **users, administrators, and inspectors**. Users can manage their accounts and water quality assessments, administrators can manage users and inspectors and monitor relevant activities, while inspectors can manage their assigned inspections.

Reports and Record Management: The system maintains important records related to users, water quality predictions, appointments, reports, and inspection details. These records help in

maintaining organized information throughout the water quality monitoring process.

Overall, **Aquaguard** integrates water quality assessment, machine learning-based prediction, safe or unsafe classification, appointment management, and inspection activities into a single web-based platform. The system provides an organized approach for monitoring water quality from multiple sources and supporting further inspection when unsafe water is identified.

CHAPTER 3

SYSTEM DESIGN AND TEST PLAN

System design refers to the process of defining the architecture, components, modules, interfaces, workflows, and data management structure of a system based on the requirements identified during the analysis phase. In the **Aquaguard – AI-Based Multi-Source Water Quality Prediction and Monitoring System**, system design focuses on developing an efficient

web-based platform for water quality assessment, machine learning-based prediction, and further inspection of unsafe water.

The system design is developed based on the functional and non-functional requirements identified during the analysis phase. It defines the interaction between users, administrators, inspectors, the machine learning model, and the database. The design supports the main objectives of Aquaguard, including water source selection, water quality parameter entry, safe or unsafe prediction, appointment booking, inspector assignment, field inspection, and record management.

Aquaguard is designed as a web-based application using **Python, Flask, HTML, CSS, JavaScript, and MySQL**. The system consists of interconnected modules such as user authentication, water source selection, water quality parameter submission, AI-based prediction, appointment management, inspector assignment, inspection management, reports, and administrator management. Each module performs a specific function and communicates with other modules to provide a smooth water quality monitoring process.

System design also includes a structured database for storing user information, water quality predictions, appointment details, reports, and inspection-related information. The backend manages communication between the frontend, machine learning model, and MySQL database. The machine learning component processes the submitted water quality parameters and provides the required prediction, while the frontend provides an organized interface for users, administrators, and inspectors.

The system design also considers proper authentication and role-based access for the different users of the system. Users can access water quality assessment and appointment-related functions, administrators can manage users and inspectors and assign inspections, and inspectors can access and update their assigned inspection details. The design ensures that the different components of Aquaguard work together efficiently and support the complete process from water quality prediction to inspection.

System design forms an important part of the software development process of Aquaguard. It acts as a bridge between the requirements analysis and implementation phases by converting the identified requirements into system architecture, modules, data flow, and processing workflows. A well-structured design helps ensure that the system is efficient, maintainable, and capable of providing reliable water quality prediction and monitoring functionality.

3.1 DATA FLOW DIAGRAM (DFD)

A data flow diagram (DFD) is a graphical representation of the flow of data within a system. It depicts how data is input, processed, stored, and output in a system or process. DFDs are widely used in software development, system analysis, and business process modelling to understand and communicate the data flow and interactions between different components of a system.

A level 0 DFD, also called a fundamental system model or a context model, represents the entire software elements as a single bubble with input and output indicated by incoming and outgoing arrows respectively. Additional process and information flow parts are represented in the next level i.e., Level 1 DFD. Each of the processes represented at Level 1 are sub-functions of the overall system depicted in the context model.

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Major components or functions are represented with Circles. Actions for input by a user or a system go in Rectangular Boxes. Databases are represented by Parallel lines enclosing a phrase corner.

3.1.1 Level 0 DFD

The Level 0 DFD of the **Aquaguard – AI-Based Multi-Source Water Quality Prediction and Monitoring System** provides a high-level representation of how data flows between the major external entities and the central system. It illustrates the interaction between the **User, Admin, and Inspector** and the Aquaguard system. The Level 0 DFD presents the overall system functionality without describing the internal sub-processes in detail.

The **User** is one of the primary external entities interacting with the Aquaguard system. The user registers and logs into the system and provides the required water quality information by selecting the water source and entering the necessary parameters. The system processes these parameters using the trained machine learning model and generates a water quality prediction. The result is provided to the user as a **safe or unsafe** classification. If the water is identified as unsafe, the user can book an appointment for further inspection.

The **Admin** acts as another major external entity in the Level 0 DFD. The admin manages users and inspectors and handles the assignment of inspectors for inspection requests. The admin can also monitor relevant appointments, inspection activities, and system records.

The required information is retrieved from and updated in the database through the system.

The **Inspector** interacts with the system after being assigned an inspection. The inspector can view the assigned inspection details and perform the required inspection. The inspection information and observations are updated in the system and stored for future reference.

The **MySQL database** plays an important role in the Level 0 DFD by storing essential information such as user details, water quality predictions, appointment details, reports, and inspection information. The database supports the different system operations by allowing required information to be stored, retrieved, and updated efficiently.

.

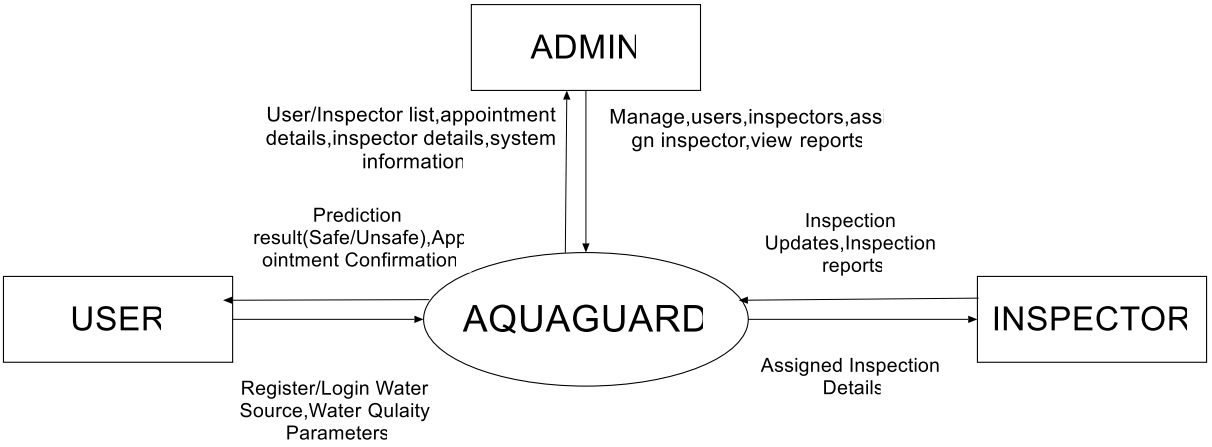


Figure 3.1: DFD Level 0

3.1.2 Level 1 DFD

The Level 1 DFD illustrates the detailed workflow and interaction between different users, functional modules, processes, and data stores within the AquaGuard AI water quality prediction and monitoring system. The diagram represents how data flows between external entities such as Users, Admin, Inspectors, and Laboratory/Water Sample sources and the different modules of the system.

The process begins with the User Management module, where users provide registration and login details for authentication. After successful login, users can access the system dashboard. User details are stored and updated in the Users database (D1). The Admin interacts with the User Management module to manage users, inspectors, and their assignments. The Admin Dashboard allows the administrator to view reports and analytics and perform administrative actions.

The Data Collection & Preprocessing module receives water sample information from manual parameter entry or laboratory/PDF reports. The collected data is cleaned and processed before being passed to the WQI Calculation & Prediction module. This module loads the trained machine learning model from the ML Models database (D2) and generates the Water Quality Index (WQI) and corresponding water quality status, such as Safe or Unsafe.

The Result Generation module receives the prediction output and generates the final water quality results and reports for the user. Prediction results and report details are stored in the Predictions database (D3). The History & Reports module retrieves previous prediction records and inspection information, allowing users to view their history and download reports in PDF format.

The Appointment Management module handles appointment requests made by users for water quality inspections. Appointment details are stored and retrieved for managing inspection schedules. The Field Inspection Management module enables inspectors to view assigned inspection details and record field inspection data. Inspectors can update inspection reports based on their observations, with inspection information maintained in the Inspections database (D5).

Overall, the Level 1 DFD clearly demonstrates the internal processes, databases, external entities, and flow of information within the AquaGuard AI system, from water quality data collection and preprocessing to WQI prediction, result generation, appointment management, field inspection, and report generation.

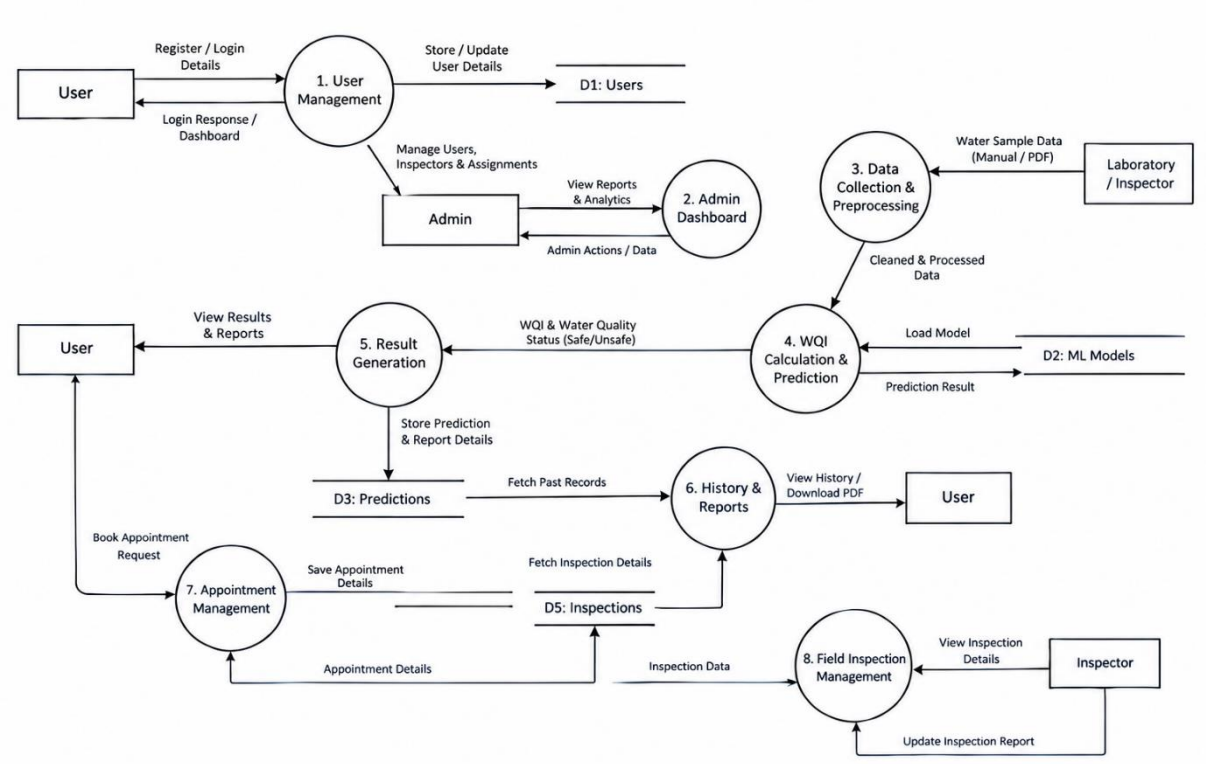


Figure 3.2: DFD Level 1

3.2 USE CASE DIAGRAM

The Use Case Diagram of the AquaGuard AI system illustrates the interaction between the primary actors of the system and the major functionalities provided by the platform. The diagram consists of three actors: the Admin, User, and Inspector. It represents how each actor interacts with different modules of the system to perform specific operations related to water quality prediction, inspection management, appointment scheduling, report generation, and system administration. The User can register, log in, submit water quality parameters, view prediction results, book inspection appointments, and view inspection reports. The Inspector can manage assigned inspections, conduct online or field inspections, enter water quality and inspection details, and generate inspection reports. The Admin manages users and inspectors, approves inspector registrations, assigns inspectors, monitors appointments and payments, and handles feedback and system activities. The use case diagram provides a high-level functional overview of AquaGuard AI and helps in understanding the responsibilities and interactions of

each user role within the water quality monitoring and management system.

3.2.1 Admin Use Case Diagram

The **Admin actor** is responsible for managing and monitoring the overall activities of the AquaGuard AI system. One of the primary use cases associated with the admin is **Manage Users**, where the admin can view, monitor, and manage registered users to maintain the proper functioning and security of the system.

Another important functionality is **Manage Inspectors**, which allows the admin to manage inspector information and monitor the inspectors registered within the system. Through the **Assign Inspectors to Users** use case, the admin can assign suitable inspectors to users who require further water quality inspection based on their prediction results.

The admin also interacts with the **View Appointment Details** use case, enabling the admin to monitor scheduled appointments between users and inspectors. This helps the admin track inspection activities, appointment status, and ensure that water quality inspection services are properly managed.

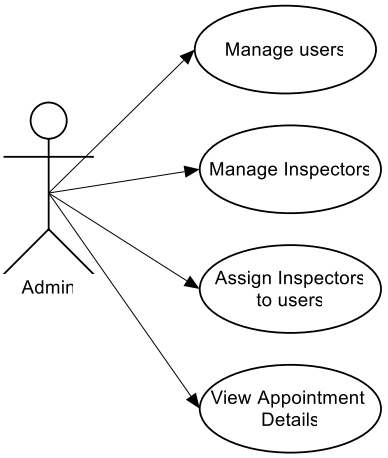


Figure 3.3: Admin Use Case Diagram

3.2.2 User Use Case Diagram

The **User actor** represents the primary user of the AquaGuard AI system who utilizes the platform to assess and monitor water quality. The user's first interaction with the system occurs through the **Enter Water Quality Parameters** use case, where the user provides relevant water quality measurements such as pH, temperature, dissolved oxygen, BOD, nitrate, and coliform values.

After entering the required parameters, the user interacts with the **Submit Water Quality Data** use case to submit the collected information for AI-based analysis. The system processes the submitted data and generates a water quality prediction based on the trained machine learning model.

The **View Prediction** use case allows the user to view the generated **Water Quality Index (WQI)** and other prediction details. Finally, through the **View Safe/Unsafe Results** use case, the user can understand whether the water is considered safe or unsafe based on the prediction, helping them make informed decisions regarding water usage and further inspection.

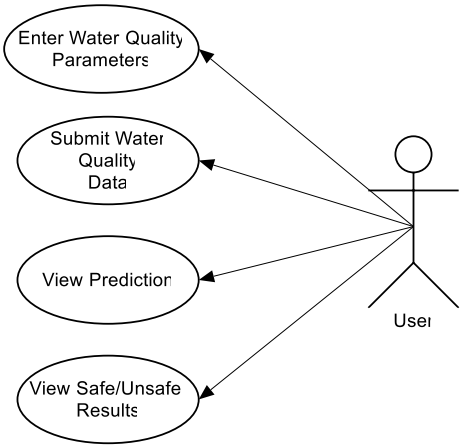


Figure 3.4: User Use Case Diagram

3.2.3 Inspector Use Case Diagram

The Inspector Module in the use case diagram allows inspectors to manage and perform water quality inspection activities within the system. The inspector can View Assigned Inspections to identify users and inspection requests assigned to them by the administrator.

The View User Details use case enables the inspector to access relevant information about the user requesting the inspection. The inspector can also use View Water Quality Details to review the user's submitted water quality parameters and prediction results before conducting the inspection.

Finally, through the Enter Inspection Observations use case, the inspector can record field inspection findings, observations, and precautionary recommendations based on the actual water quality assessment. These details can then be used for generating and maintaining the user's inspection report.

.

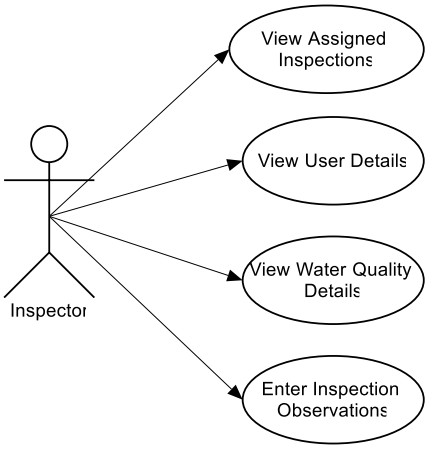


Figure 3.5: Doctor Use Case Diagram

3.3 TABLE DESIGNS

3.3.1 User Table

- Stores and manages water quality data, inspection records, prediction results, and monitoring information to assess water safety and support informed water quality management.

Table 3.1: user table

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for each user
name	Varchar(150)	Unique, Not Null	Unique username or login name of the user
email	Varchar(255)	Unique, Not Null	User's email address
password	Varchar(255)	Not Null	Encrypted password of the user
Created_at	timestamp	Current Time Stamp	Time at which it created
role	Varchar(50)	Not Null	Current Role(User,Inspector,Admin)

Phone_number	Varchar(50)	Not Null	Phone Number of the user
Profile_image	Varchar(255)	Not Null	Image of the User
location	Varchar(150)	Not Null	Location of the user

3.3.2 Inspector Table

Stores detailed water quality parameters and inspection information used to evaluate and monitor the safety and quality of water sources.

Table 3.2: Inspector table

Field	Data Type	Constraints	Description
Inspector_id	Integer	Primary Key, Auto Increment	Unique identifier for each Inspector
Full_name	Varchar(20)	Foreign Key, Not Null	Full name of the Inspector
email	Varchar(20)	Not Null	Email id of the Inspector
phone	Varchar(20)	Null	Contact number
password	Varchar(10)	Nullable	Encrypted password of the Inspector
district	Varchar(15)	Null	District name
designation	Varchar(20)	Nullable	Designation of the Inspector
status	enum	Nullable	Status of the Inspector(Active,Inactive)
created_at	Time Stamp	Default Current Timestamp	Time at which it created

3.3.3 Predictions Table

Stores water quality prediction results, including the measured parameters, predicted WQI values, and corresponding water safety classifications.

Table 3.3: Predictions table

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for user doctor
user_id	Integer	Foreign Key, Not Null	References the user in the user table
Water_source	Varchar(50)	Not Null	Types of Water resources/Source(River,Tap,Industrial,etc)
Temperature	float	Not Null	Temperature of the Water
Dissolved_oxygen	float	Default = 0	Dissolved oxygen of the water
Ph	float	Not Null	Ph of the Water
bod	float	Nullable	Bod of the Water
Prediction	Varchar(20)	Not Null	Prediction of Water based on WQI(Safe,Unsafe)
Pollution_level	Varchar(50)	Not Null	Level of pollution(High pollution,Low pollution,safe)

3.3.4 Payments Table

· Stores payment transaction details associated with water quality inspection and related services, including payment status and transaction information.

Table 3.4: payments table

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for each user payment
User_id	Integer	Not Null	References the user in

			the user table
amount	Integer	Not Null	Total Amount user paid
status	Varchar(50)	Not Null	Status of the payment(Paid,Unpaid)
Appointment_id	Int(11)	Unique, Not Null	Appointment id of the user
Payment_method	Varchar(50)	Nullable	Method of payment(Gpay,Phonepe)
created_at	Time Stamp	Default Current Timestamp	Time at which payment is done

3.3.5 inspection_reports Table

· Stores detailed water quality inspection reports, including inspection findings, observed parameters, safety status, recommendations, and report-related information.

Table 3.5: inspection_reports table

Field	Data Type	Constraints	Description
Report_id	Integer(11)	Primary Key, Auto Increment	Unique identifier for each prediction Report
Appointment_id	Integer(11)	Foreign Key, Not Null	Id of the Appointment
findings	text	Not Null	Findings during Testing Water
recommendations	Text	Not Null	Recommendation after Testing
Inspection_date	Date	Not Null	Date at which the Inspection is done
Report_file	Varchar(255)	Nullable	File of the Report after final testing is done

3.3.6 field_inspection Table

Stores field inspection details, including inspection observations, precautionary measures, inspector remarks, and the water quality assessment status.

Table 3.6: field_inspection table

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for each appointment
inspector_id	Integer	Foreign Key, Not Null	Id of the Inspector
user_id	Integer	Foreign Key, Not Null	Id of the user
appointment_id	Integer	Not Null	Id of the Appointment
Water_source	Varchar(100)	Not Null	Source of the water(Tap,River,Well etc)
Water_appearance	Varchar(100)	Not Null	Appearance of the water dyring field Inspection
Inspection_date	DateTime	Default Current Timestamp	Date at which the Inspection is done

3.3.7 Appointments Table

· Stores appointment details between users and inspectors for scheduling and managing water quality inspections, including appointment date, time, status, and related information.

Table 3.7: appointments table

Field	Data Type	Constraints	Description
Appointment_id	Integer	Primary Key, Auto Increment	Unique identifier for each Appointment
user_id	Integer	Foreign Key, Not Null	References the user table

inspector_id	Integer	Foreign Key, Not Null	References the inspector table
Appointment_date	Text	Not Null	Date of the Appointment
Appointment_time	Time	Not Null	Time of the Appointment
Address	Text	Nullable	Address of the User
Issue_description	Text	Not Null	Issues of the Water

3.3.8 Feedbacks Table

Stores user feedback and comments regarding water quality inspections, prediction results, and the overall AquaGuard AI system experience.

Table 3.8: feedbacks table

Field	Data Type	Constraints	Description
Feedback_id	Integer	Primary Key, Auto Increment	Unique identifier for each Feedback
appoitment_id	Integer	Foreign Key, Not Null	Id of the Appointment
User_id	Integer	Not Null	Id of the User
rating	Integer	Not Null	Ratings done by User
categories	Varchar(500)	Not Null	Categories(Punctuality,Communication etc)
comments	text	Not Null	Comments done by the user

3.4 PERFORMANCE REQUIREMENTS

The performance requirements of the **Aquaguard AI system** are essential to ensure efficient operation, fast response, reliability, and scalability. The system should provide quick response times for important operations such as **user authentication, water quality prediction, appointment booking, inspection management, and report generation**. The machine

learning model should process submitted water quality parameters efficiently and provide prediction results without noticeable delays.

The system should be capable of supporting **multiple users simultaneously** without significant performance degradation. It should efficiently handle increasing numbers of users, water quality records, predictions, appointments, and inspection requests. The database should provide efficient storage, searching, updating, and retrieval of system data to maintain smooth operation as the amount of data increases.

The machine learning module should execute efficiently to generate water quality predictions within a reasonable processing time. The system should also provide timely updates for **prediction results, appointment status, inspector assignments, inspection reports, and notifications**. Communication between the frontend, backend, database, and machine learning model should be optimized for reliable and efficient data exchange.

The system should maintain high reliability with **proper error handling, data backup, and recovery mechanisms** to minimize service interruptions. The web interface should remain responsive across different devices and network conditions. Overall, the system should provide a **fast, reliable, scalable, and user-friendly experience** for water quality assessment and monitoring.

3.5 SOFTWARE QUALITY ATTRIBUTES

The software quality attributes of the **Aquaguard AI system** ensure that the platform provides reliable, secure, efficient, and user-friendly water quality monitoring services.

Reliability ensures that the system consistently performs important operations such as water quality prediction, appointment scheduling, inspection management, report generation, and user management without frequent failures or interruptions.

Usability focuses on providing a simple and intuitive interface for users, inspectors, and administrators. Clear navigation, organized dashboards, understandable prediction results, and simple forms allow users to operate the system without requiring advanced technical knowledge.

Security is essential for protecting user information, water quality records, inspection details, and system data. The system should implement secure authentication, password hashing, role-based access control, HTTPS communication, and appropriate database security measures to prevent unauthorized access.

Performance ensures that the system provides quick response times for water quality

prediction, data processing, appointment booking, inspection updates, and report generation. The system should efficiently handle multiple users and data requests without significant performance degradation.

Scalability ensures that the system can accommodate an increasing number of users, water quality records, predictions, appointments, and inspections. The architecture should also support future improvements and integration with additional water monitoring services.

Maintainability allows developers to easily identify errors, update machine learning models, modify existing features, and add new functionalities. A modular and well-structured design with properly organized code improves the long-term maintenance of the system.

Compatibility ensures that the Aquaguard AI web application works effectively across different browsers, operating systems, and devices such as desktops, laptops, tablets, and smartphones while maintaining a consistent user experience.

Availability ensures that the system remains accessible whenever required. Proper database backups, error handling, recovery mechanisms, and reliable server operation should be used to minimize downtime and prevent data loss.

Finally, **testability** ensures that the system can be properly evaluated through unit testing, integration testing, system testing, security testing, performance testing, and user acceptance testing. Testing helps ensure that the water quality prediction, authentication, appointment, inspection, reporting, and administrative modules function correctly and reliably.

3.6 DETAILED DESIGN

The detailed design of the **Aquaguard AI system** provides a structured approach to system architecture, water quality prediction, database management, user interaction, inspection management, reporting, security, and machine learning integration. The design ensures that the system satisfies its functional and non-functional requirements while remaining reliable, maintainable, and scalable.

3.6.1 System Architecture

The **Aquaguard AI system** is developed using a web-based architecture with a frontend, backend, database, and machine learning modules. The frontend provides an interactive interface through which users, inspectors, and administrators can access their respective functionalities.

The backend manages the main application logic and coordinates communication between the frontend, database, machine learning models, inspection modules, appointment system, and reporting components. The database stores user details, water quality parameters, prediction results, appointments, inspector information, and inspection reports. The modular architecture allows the system to be easily maintained and extended with additional water quality monitoring features.

3.6.2 User Registration and Authentication

Users can register using valid account details such as email and password. The system securely manages authentication and user sessions to prevent unauthorized access. Passwords are stored using secure hashing techniques.

Role-based access control is implemented to provide different functionalities to **users, inspectors, and administrators**. Users can access water quality prediction and inspection-related services, inspectors can manage assigned inspections, and administrators can manage users, inspectors, appointments, and overall system activities.

3.6.3 User Profile and Water Quality Management

Users can create and update their profiles through a dedicated profile interface. They can select the required **water source**, enter water quality parameters, and submit the information for analysis.

The system maintains prediction history, appointment details, inspection information, and generated reports. Users can view previous prediction results and access relevant recommendations based on the analyzed water quality data.

Inspectors can manage their profiles and view assigned inspection requests. Administrators can manage user and inspector information through the administrative dashboard.

3.6.4 AI-Based Water Quality Prediction

The system uses **machine learning models** to analyze water quality parameters and predict the quality status of the submitted water sample. Users can select different water sources such as **well, river, tap, lake, and industrial water** and provide the required parameters.

The prediction module processes the input data using the trained machine learning model and

generates the predicted water quality result. The system can also calculate or display the **Water Quality Index (WQI)** and provide suitable recommendations based on the prediction.

3.6.5 Water Quality Data Processing and Evaluation

Water quality parameters are entered through structured forms or obtained from supported laboratory report uploads. The backend validates and processes the submitted data before sending it to the machine learning model.

The prediction system applies the required preprocessing techniques and uses the trained model to generate results. The processed information is evaluated to determine the water quality condition and is then returned to the user in an understandable format. Historical prediction data can also be maintained for future reference and monitoring.

3.6.6 Reporting, Inspection and Monitoring

The system generates water quality reports containing the submitted parameters, prediction results, WQI information, recommendations, and other relevant details. Reports can be accessed by authorized users for future reference.

When further verification is required, users can request an **online or field inspection**. Administrators can assign inspectors to inspection requests, while inspectors can review the submitted information, conduct inspections, record observations and measured parameters, and submit inspection reports.

The administrator dashboard provides monitoring information such as the number of users, predictions, appointments, inspections, and safe or unsafe water results.

3.6.7 Security Measures

The system implements security measures to protect user information, water quality records, inspection details, and authentication data. Secure password storage, role-based access control, session management, input validation, and HTTPS communication are used to reduce unauthorized access and security risks.

Access to sensitive operations such as profile modification, inspection updates, report access, and administrative functions is restricted according to user roles. Database access is also controlled to maintain the confidentiality and integrity of stored information.

3.6.8 Performance Optimization

The Aquaguard AI system is designed to provide fast response times during **water quality prediction, data processing, appointment booking, inspection management, and report generation**. Efficient database operations help improve the retrieval and storage of user, prediction, appointment, and inspection data.

The machine learning model is integrated with the backend to generate predictions efficiently. The modular architecture allows the system to handle increasing users and records while maintaining acceptable performance and responsiveness.

3.6.9 Testing and Deployment

Comprehensive testing is performed to ensure the functionality, reliability, security, and performance of the Aquaguard AI system. **Unit testing, integration testing, system testing, performance testing, security testing, and user acceptance testing** can be used to validate the major modules.

Special attention is given to testing the machine learning prediction module, water quality data processing, authentication, appointment management, inspection workflow, report generation, and administrative functions. The system can be deployed on a local or cloud-based server environment, allowing future improvements and additional water quality monitoring features to be incorporated efficiently.

3.7 TEST CASES

3.7.1 Test Case for Admin

Category	Test Case	Description	Expected Result
Authentication	Admin Login	Ensure the admin can log in using valid admin credentials.	The admin is successfully authenticated and redirected to the admin dashboard.
User Management	View All Users	Ensure the admin can view all registered	A complete list of users with name, email, role, and status

		users.	is displayed.
Inspector Managemen	View Inspector	Ensure the admin can add a new inspector.	A list of inspectors and their details is displayed.
Inspector Management	Add Inspector	Ensure the admin can add a new inspector..	The inspector is successfully registered and added to the inspector list.
Inspector Management	Assign Inspector	Ensure the admin can assign an inspector to an inspection request.	Ensure the admin can assign an inspector to an inspection request.
Inspection Management	View Inspection Requests	Ensure the admin can viewsubmitted inspection requests.	All inspection requests with their current status are displayed.
Inspection Management	Manage Inspection Status	Ensure the admin can monitor inspection status.	The inspection status is displayed and updated correctly..
Water Quality Management	View Water Quality Results	Ensure the admin can view water quality results.	Parameters, WQI, and Safe/Unsafe results are displayed correctly.
Report Management	View Reports	Ensure the admin can view generated inspection reports.	Generated reports are displayed and can be viewed/downloaded..
Security	Unauthorized Admin Access	Ensure normal users cannot access admin pages.	Access is denied to unauthorized users.
Authentication	Admin Logout	Ensure the admin can log out of the system.	Admin session ends and the login page is displayed..

3.7.2 Test Case for User

Category	Test Case	Description	Expected Result
Authentication	User Registration		The user account is created successfully.

		Ensure a new user can register by entering valid details.	
User Authentication	User Login	Ensure a registered user can log in with valid credentials.	The user is successfully logged in and redirected to the user dashboard
User Authentication	Invalid Login	Verify login using incorrect credentials.	An appropriate error message is displayed and access is denied.
User Profile	View Profile	Ensure the user can view their profile details.	The user's name, email, role, and other available details are displayed correctly.
User Profile	Update Profile	Ensure the user can update their profile information.	The updated information is saved successfully.
WQI Calculation	Calculate WQI	Ensure the system calculates the Water Quality Index.	The WQI value is calculated and displayed correctly.
Inspection Management	Book Inspection	Ensure the user can book an inspection when required.	The inspection request is successfully submitted.
Inspection Management	View Inspection Status	Ensure the user can track the inspection status.	The current status, such as Pending, Assigned, or Completed, is displayed.
Reports	View Inspection Report	Ensure the user can view the completed inspection report.	Therelevant inspection details, parameters, WQI, and prediction result are displayed.
Reports	Download PDF Report	Ensure the user can download the generated report.	The inspection report issuccessfully generatedand downloaded as a PDF.

History	View Previous Inspections	Ensure the user can view previous inspection records.	Previous inspection details and results are displayed correctly.
Notifications	View Inspection Updates	Ensure the user receives updates regarding inspection requests..	Relevant inspection status or result notifications are displayed.
Authentication	User Logout	Ensure the user can log out of the system.	The session is terminated and the user is redirected to the login page.

3.7.3 Test Case for Inspector

Category	Test Case	Description	Expected Result
Authentication	Inspector Login	Ensure the inspector can log in using valid credentials.	The inspector is successfully logged in and redirected to the inspector dashboard.
Authentication	Invalid Login	Verify login using incorrect inspector credentials.	The inspector is successfully logged in and redirected to the inspector dashboard.
Inspector Profile	View Profile	Ensure the inspector can view their profile details.	The inspector's profile information is displayed correctly.
Inspection Management	View Assigned Inspections.	Ensure the inspector can view inspections assigned by the admin.	All assigned inspection requests are displayed with relevant user and inspection details.
Field Inspection	Conduct Field Inspection	Ensure the inspector can conduct a field inspection.	The inspector can enter the required field inspection details and observations

			successfully.
WQI Calculation	Calculate WQI	Ensure the system calculates the Water Quality Index from the inspection parameters.	The system clearly displays whether the water is Safe or Unsafe..
Inspection History	View Completed Inspections	Ensure the inspector can viewpreviously completed inspections.	Completed inspection records are displayed correctly.

CHAPTER 4
IMPLEMENTATION AND RESULTS

4.1 IMPLEMENTATION

The implementation of **AquaGuard AI – AI-Based Multi-Source Water Quality Prediction and Monitoring System** began with a detailed analysis of the requirements necessary for developing an intelligent, reliable, and efficient water quality monitoring platform. The project was designed to provide users, inspectors, and administrators with an integrated environment for water quality assessment using Artificial Intelligence, Machine Learning, Water Quality Index (WQI) calculation, and modern web technologies. The primary objective of the implementation phase was to develop a user-friendly and scalable system capable of performing water quality prediction, online and field inspection, water quality analysis, inspection management, and automated report generation effectively.

The system was developed using **Python and the Flask framework** because of their flexibility, lightweight architecture, and strong support for Machine Learning integration. **MySQL** was used as the database management system for storing user information, inspector details, inspection requests, water quality parameters, prediction results, WQI values, inspection observations, and report information. The implementation focused on developing a modular architecture that ensures smooth communication between the frontend interface, Flask backend, Machine Learning prediction modules, WQI calculation module, and database.

4.1.1 Backend Development

The backend of AquaGuard AI was developed using **Python and Flask**. The backend manages the core functionalities of the system, including user authentication, role-based access, user management, inspector management, inspection requests, water quality parameter processing, Machine Learning prediction, WQI calculation, report generation, and database operations. MySQL was integrated as the database management system for efficient storage, retrieval, and processing of water quality and user-related information.

Secure authentication and authorization mechanisms were implemented to provide different levels of access for **users, inspectors, and administrators**. Role-based access control ensures that each type of user can access only the functionalities assigned to their role. Users can submit

water quality information and request inspections, inspectors can conduct assigned inspections and enter field observations, and administrators can manage users, inspectors, and inspection activities.

Machine Learning modules were integrated into the backend to process water quality parameters and generate water quality predictions. The system accepts parameters such as **Temperature, Dissolved Oxygen, pH, BOD, Faecal Streptococci, Nitrate, Faecal Coliform, Total Coliform, and Conductivity**. The input data is preprocessed before being provided to the trained Machine Learning model. Missing values are handled using appropriate imputation techniques to improve the reliability of the prediction process.

The backend also performs **Water Quality Index (WQI) calculation** to provide an overall numerical representation of water quality. Based on the processed parameters, Machine Learning prediction, and WQI value, the system determines the final water quality status as **Safe or Unsafe**. The processed inspection details and prediction results are stored in the MySQL database for future reference and report generation.

4.1.2 Frontend Development

The frontend of AquaGuard AI was designed to provide an interactive, responsive, and user-friendly water quality monitoring experience. The user interface was developed using **HTML, CSS, and JavaScript**, allowing users, inspectors, and administrators to access system functionalities through separate interfaces.

Users can register and log in to the system, select the required water source, enter water quality parameters, and perform an online inspection. The system supports multiple water sources such as **Well, River, Tap, Lake, and Industrial water**. After submitting the required parameters, the system processes the information and displays the calculated WQI, water quality prediction, and final Safe/Unsafe status.

The system also provides a **field inspection facility** through which users can request an inspection when required. Assigned inspectors can view inspection requests, enter measured water quality parameters, provide observations, and add precautionary methods based on the inspection results. This information is then processed and stored for further analysis.

The admin interface was also implemented as part of the frontend. It enables administrators to manage registered users and inspectors, view inspection requests, assign inspectors, monitor inspection status, and access water quality results and generated reports through a centralized dashboard.

4.1.3 Integration and Testing

After completing the frontend and backend development, both components were integrated to ensure seamless communication and proper functioning of the AquaGuard AI system. The Machine Learning prediction module, WQI calculation module, MySQL database, inspection modules, and report generation functionality were integrated with the Flask backend.

Extensive testing procedures were conducted to validate the **functionality, reliability, security, accuracy, and usability** of the system. The testing process included unit testing, integration testing, white-box testing, black-box testing, security testing, and usability testing. These testing procedures helped identify and resolve errors in different modules of the application.

White-box testing focused on verifying the internal logic of the system, including backend processing, database operations, Machine Learning prediction, WQI calculation, authentication, and report generation. Important components such as parameter validation, missing-value handling, prediction processing, inspection workflows, and database transactions were tested to ensure correct system behavior. Different input conditions, including invalid values, missing parameters, and incomplete inspection details, were also evaluated.

Black-box testing was conducted from the perspective of end users without considering the internal implementation of the system. Functional testing was performed to verify major features such as user registration, login, water quality parameter entry, online inspection, field inspection requests, water quality prediction, WQI calculation, Safe/Unsafe classification, inspector assignment, and report generation. Boundary and invalid input conditions were also tested to ensure that the system provided appropriate validation messages and expected results.

Usability testing was conducted to evaluate the user interface, navigation, form design, result presentation, and overall user experience. The testing process focused on ensuring that users could easily enter water quality parameters, understand the prediction results, access inspection information, and view generated reports. The inspector and administrator interfaces were also evaluated to ensure that inspection management and monitoring activities could be performed efficiently.

The combined results from white-box testing, black-box testing, integration testing, security testing, and usability testing confirmed that **AquaGuard AI** functions reliably and efficiently. The successful integration of the frontend, Flask backend, MySQL database, Machine Learning models, WQI calculation, inspection management, and PDF report generation ensured that the system was ready for practical use as an intelligent water quality prediction and monitoring

platform.

4.2 IMPLEMENTATION RESULTS

The implementation of **AquaGuard AI – AI-Based Multi-Source Water Quality Prediction and Monitoring System** resulted in an intelligent and efficient platform for analyzing and monitoring water quality. By utilizing Python, Flask, Machine Learning algorithms, WQI calculation techniques, MySQL, and modern web technologies, the project successfully established communication between the frontend interface, backend services, Machine Learning prediction modules, database, inspection modules, and report generation system. The implementation of role-based access control provided separate functionalities for users, inspectors, and administrators and ensured secure access to the system.

The system successfully enabled users to perform water quality assessments by entering important water quality parameters such as **Temperature, Dissolved Oxygen, pH, BOD, Faecal Streptococci, Nitrate, Faecal Coliform, Total Coliform, and Conductivity**. The system processes the entered values and uses the trained Machine Learning model to predict water quality. The **Water Quality Index (WQI)** is also calculated to provide an overall assessment of the water condition. Based on the prediction and water quality assessment, the system displays the final result as **Safe or Unsafe**, helping users understand the quality of the tested water.

The implementation of online and field inspection modules provided flexible methods for water quality assessment. Users can perform online inspections by entering the required water quality parameters, while field inspections allow inspectors to enter measured parameters, inspection observations, and precautionary methods. This integration provides a systematic approach to collecting and analyzing water quality information from different sources.

The system supports multiple water sources, including **Well, River, Tap, Lake, and Industrial water**. The water quality parameters submitted during inspections are processed by the backend and stored in the MySQL database. The stored information can be retrieved for monitoring, inspection history, analysis, and report generation. This provides an organized method for maintaining water quality records and monitoring previous inspection results.

The **admin module** effectively facilitates user management, inspector management, inspection request monitoring, inspector assignment, water quality result monitoring, and

report management through a centralized dashboard. Administrators can view registered users and inspectors, manage inspection requests, assign inspectors, and monitor the progress of inspections. The inspector module allows assigned inspectors to view inspection requests, enter field measurements and observations, update inspection status, and contribute information required for generating the final inspection report.

The report generation module successfully produces detailed **PDF inspection reports** containing relevant user and inspection information, water quality parameters, WQI values, prediction results, Safe/Unsafe status, and inspector observations where applicable. These reports provide users and administrators with a structured record of the water quality assessment and can be used for future reference and monitoring purposes.

Comprehensive testing and system optimization ensured that AquaGuard AI provided stable performance, reliable data processing, secure user access, and responsive operation. Functional testing, integration testing, white-box testing, black-box testing, security testing, and usability testing were conducted to verify the major system components. The testing results confirmed that the implemented modules performed their intended functions correctly and that the system could handle different valid and invalid input conditions effectively.

The modular architecture of AquaGuard AI also provides opportunities for future expansion. The system can be enhanced with **real-time water quality monitoring, IoT-based sensors, additional water quality parameters, larger and more diverse datasets, advanced Machine Learning models, cloud-based deployment, mobile applications, and real-time alerts**. These enhancements can improve the accuracy, scalability, and practical applicability of the system in different water quality monitoring environments.

Overall, **AquaGuard AI successfully achieved its objectives of providing an intelligent, user-friendly, and efficient water quality prediction and monitoring platform**. The system effectively combines Machine Learning, WQI calculation, web technologies, database management, online and field inspection, and automated report generation to support water quality assessment. By providing clear Safe/Unsafe results and detailed inspection reports, AquaGuard AI offers a practical approach to water quality monitoring and provides a strong foundation for future development and real-world implementation.

CHAPTER 5

CONCLUSION AND FUTURE WORKS

5.1 CONCLUSION

The AquaGuard AI – AI-Based Multi-Source Water Quality Prediction and Monitoring System successfully demonstrates the application of Artificial Intelligence, Machine Learning, and modern web technologies for improving water quality assessment and monitoring. The project was developed to provide an efficient, intelligent, and user-friendly platform where users, inspectors, and administrators can perform and manage water quality inspections, analyze water quality parameters, calculate Water Quality Index (WQI), and obtain Safe/Unsafe water quality predictions.

The system effectively supports water quality prediction, WQI calculation, online inspection, field inspection, inspection management, water source monitoring, and automated report generation. Users can enter important water quality parameters and obtain an overall assessment of the tested water. The Machine Learning model processes the input parameters and provides a prediction, while the WQI calculation provides a numerical representation of water quality. The final result is presented as Safe or Unsafe, making the water quality assessment easier to understand.

AquaGuard AI supports multiple water sources, including Well, River, Tap, Lake, and Industrial water. The system provides both online and field inspection facilities, allowing water quality information to be collected through different inspection methods. Inspectors can enter measured parameters, observations, and precautionary methods during field inspections. This provides a structured approach to collecting and analyzing water quality information.

The use of Python and Flask enabled the development of a lightweight, flexible, and reliable web-based architecture capable of integrating Machine Learning models with web application functionalities. MySQL provides structured storage for user details, inspector information, inspection requests, water quality parameters, prediction results, WQI values, and report information. The role-based system provides separate functionalities for users, inspectors, and administrators, supporting organized system management and controlled access.

The project successfully achieved its primary objectives of providing intelligent water quality prediction, WQI-based assessment, online and field inspection, inspection monitoring, and automated report generation. The implementation of Machine Learning reduces the dependency on purely manual analysis and provides a faster method for evaluating water

quality based on the available parameters. The generated PDF reports also provide a convenient and structured record of inspection results for future reference.

Comprehensive testing confirmed the functionality, reliability, usability, and security of the major system components. The testing process covered user authentication, water quality parameter validation, prediction, WQI calculation, inspection management, inspector activities, database operations, report generation, and role-based access. The successful integration of these modules demonstrates that AquaGuard AI can serve as a practical foundation for intelligent water quality monitoring.

Overall, AquaGuard AI successfully combines Machine Learning, WQI calculation, web technologies, database management, online and field inspection, and automated reporting into a single water quality monitoring platform. The project provides an effective approach for assessing water quality from multiple sources and helps users understand whether the tested water is safe or unsafe. It also establishes a strong foundation for future enhancements involving real-time monitoring, IoT, advanced Machine Learning, and large-scale water quality analysis.

5.2 FUTURE WORKS

Looking ahead, there are several opportunities for enhancing the functionality, accuracy, scalability, and intelligence of the AquaGuard AI system. Future developments can focus on improving water quality prediction accuracy, expanding the range of water quality parameters, integrating real-time monitoring technologies, enhancing inspection capabilities, and supporting large-scale water quality analysis. By incorporating advanced Artificial Intelligence, Machine Learning, IoT, and cloud technologies, AquaGuard AI can be further developed into a comprehensive real-time water quality monitoring platform.

5.2.1 Future Implementation Plan

- **Integrate advanced Machine Learning and Deep Learning models** to improve the accuracy and reliability of water quality prediction and classification.
- **Implement real-time water quality monitoring** by integrating IoT-based sensors capable of continuously measuring parameters such as pH, temperature, dissolved oxygen, conductivity, turbidity, and other water quality indicators.

- **Expand the water quality parameters** supported by the system to provide a more comprehensive assessment of water quality and improve prediction reliability.
- **Enhance the dataset** by collecting larger, more diverse, and location-specific water quality datasets from different geographical regions and water sources. This can improve the generalization and accuracy of the Machine Learning models.
- **Improve field inspection capabilities** by introducing GPS-based location tracking, digital inspection forms, image-based evidence collection, and improved inspector reporting features.

Overall, the future enhancements of **AquaGuard AI** will focus on improving prediction accuracy, expanding water quality analysis, enabling real-time monitoring, enhancing field inspection capabilities, and increasing system scalability. The integration of IoT, advanced Machine Learning, cloud computing, GIS, mobile technologies, and larger datasets can transform AquaGuard AI into a more comprehensive and intelligent water quality monitoring solution suitable for wider real-world applications.

REFERENCES

Conference Proceedings:

1. World Health Organization. (2022). Guidelines for Drinking-water Quality: Fourth Edition Incorporating the First and Second Addenda. World Health Organization..
2. United States Environmental Protection Agency. (2009). National Primary Drinking Water Regulations. U.S. Environmental Protection Agency.

Journal Articles:

1. Brown, R. M., McClelland, N. I., Deininger, R. A., & Tozer, R. G. (1970). A Water Quality Index—Do We Dare? *Water & Sewage Works, 117*(10), 339–343..
2. Horton, R. K. (1965). An Index-Number System for Rating Water Quality. *Journal of the Water Pollution Control Federation, 37*(3), 300–306.
3. Tyagi, S., Sharma, B., Singh, P., & Dobhal, R. (2013). Water Quality Assessment in Terms of Water Quality Index. *American Journal of Water Resources, 1*(3), 34–38.

4. Abbasi, T., & Abbasi, S. A. (2012). Water Quality Indices Based on Physicochemical Parameters: A Review. *Journal of Water Health, 10*(1), 1–16.

APPENDIXES

Appendix A- Screenshots

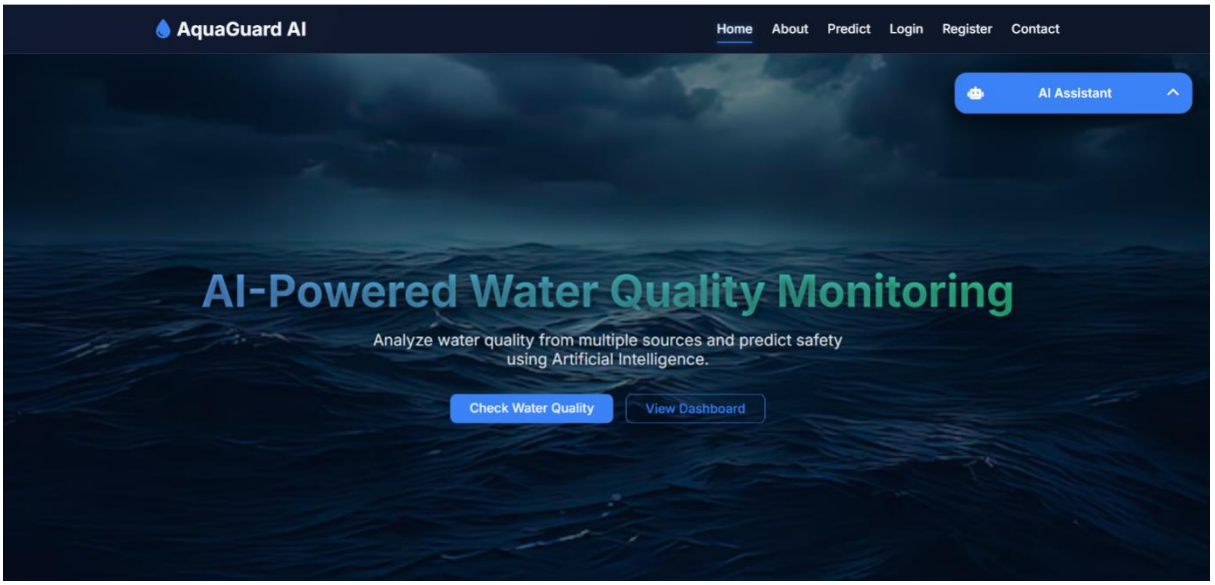


Figure A.1 Home Page

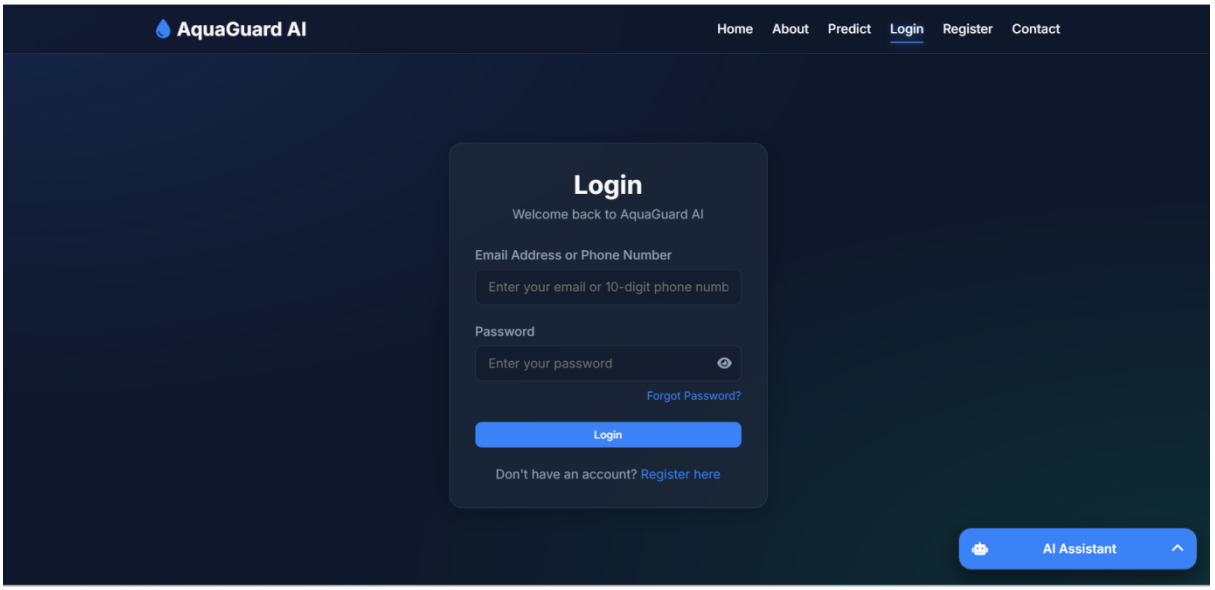


Figure A.2 Login Page

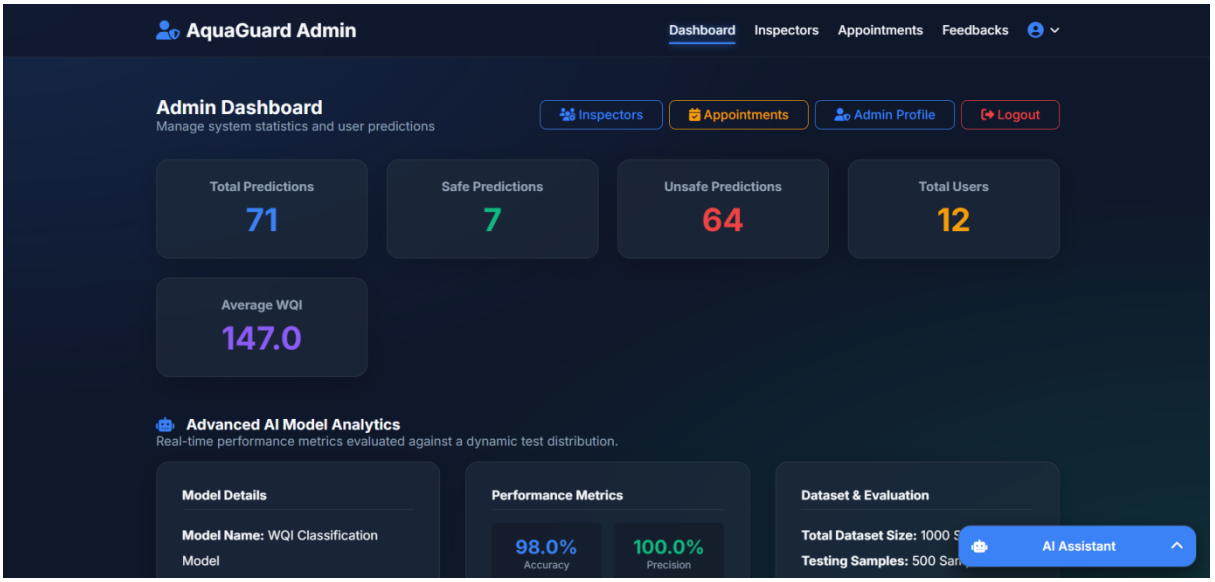


Figure A.3: Admin Dashboard

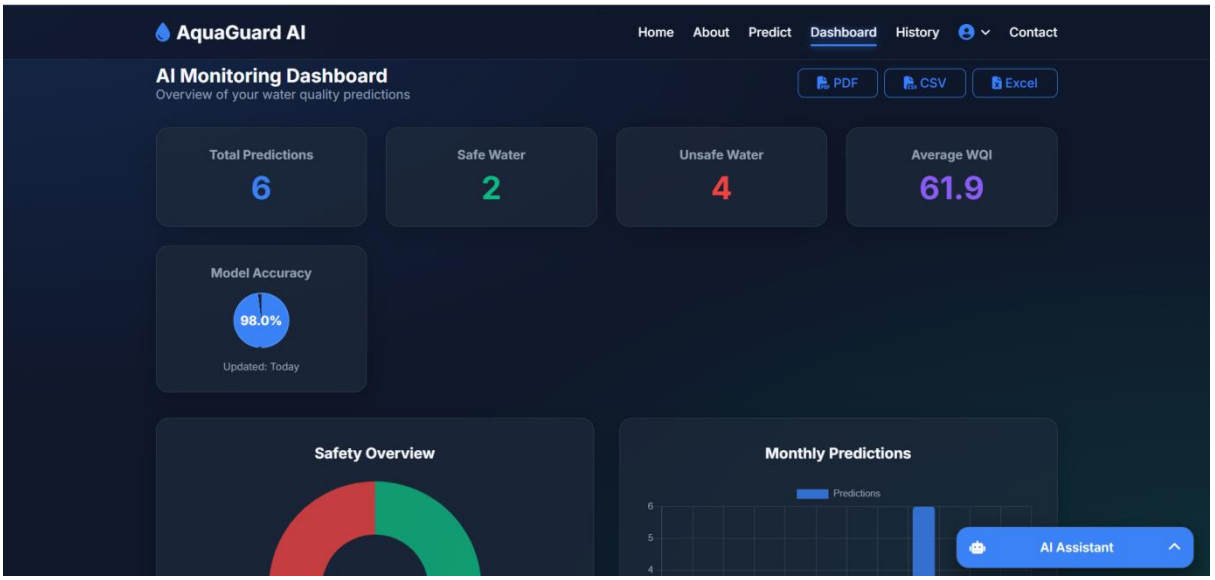


Figure A.4: User Dashboard

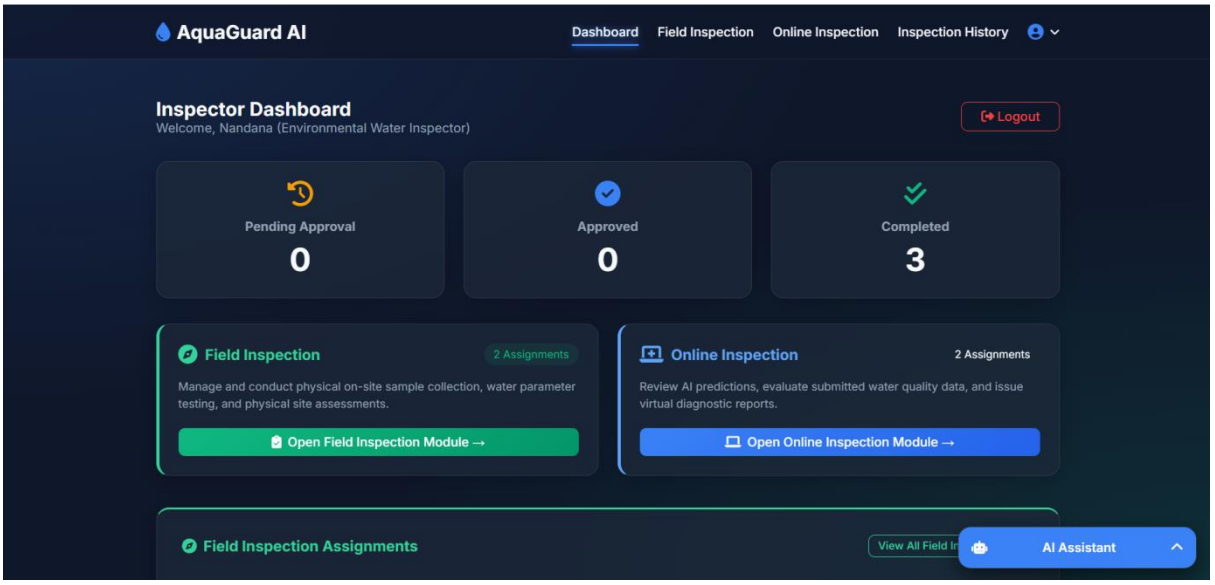


Figure A.5: Inspector Dashboard

Appendix B- Sample Code

APP.PY

```
import os
import json
import pickle
import numpy as np
from datetime import datetime, timedelta, date, time
from decimal import Decimal

def make_json_serializable(obj):
    if obj is None:
        return None
    if isinstance(obj, dict):
        return {k: make_json_serializable(v) for k, v in obj.items()}
    elif isinstance(obj, list):
        return [make_json_serializable(v) for v in obj]
    elif isinstance(obj, (datetime, date, time, timedelta)):
        return str(obj)
    elif isinstance(obj, Decimal):
        return float(obj)
```

```
    elif isinstance(obj, bytes):
        return obj.decode('utf-8', errors='ignore')
    return obj

from flask import Flask, render_template, request, redirect, url_for, session, flash, jsonify, Response
import csv
import io
import traceback
import mysql.connector
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from dotenv import load_dotenv
from groq import Groq
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
import re
import hashlib
import secrets
from flask_wtf.csrf import CSRFProtect
from email_validator import validate_email, EmailNotValidError
from flask_mail import Mail, Message
load_dotenv(override=True)

from typing import Any
imputer: Any = None
model: Any = None

# Debug print to verify variables are loaded
print("Loaded DB:", os.getenv("MYSQL_DATABASE"))

def calculate_wqi(do, ph, bod, nitrate, cond):
    score = (abs(ph-7)*10) + (bod*5) + (nitrate*2) + (cond/100)
```

```
    if do < 5: score += 20
    if score < 26: cat = "Excellent"
    elif score < 51: cat = "Good"
    elif score < 76: cat = "Poor"
    elif score < 101: cat = "Very Poor"
    else: cat = "Unsuitable"
    return round(score, 1), cat

def get_model_metrics():
    if imputer is None or model is None:
        return {"accuracy": 0, "precision": 0, "recall": 0, "f1": 0, "conf_matrix": [[0,0],[0,0]],
"class_report": {}, "total_samples": 0, "test_samples": 0, "n_features": 0}




        if not text.strip():
            return jsonify({"error": "Could not extract readable text from PDF"}), 400

        # Regex patterns to find parameters
        params = {
            "temperature": r"(?i)temperature\s*\(?°c\)?\s*[:\-=]?\s*([\d.]+)",
            "do": r"(?i)(?:dissolved oxygen|do)\s*\(?mg/l\)?\s*[:\-=]?\s*([\d.]+)",
            "ph": r"(?i)ph(?: value| level)?\s*[:\-=]?\s*([\d.]+)",
            "bod": r"(?i)bod\s*\(?mg/l\)?\s*[:\-=]?\s*([\d.]+)",
            "faecal_streptococci": r"(?i)fa?ecal streptococci\s*\(?mpn/100ml\)?\s*[:\-
=]?\s*([\d.]+)",
            "nitrate": r"(?i)nitrate\s*\(?mg/l\)?\s*[:\-=]?\s*([\d.]+)",
            "faecal_coliform": r"(?i)fa?ecal coliform\s*\(?mpn/100ml\)?\s*[:\-=]?\s*([\d.]+)",
            "total_coliform": r"(?i)total coliform\s*\(?mpn/100ml\)?\s*[:\-=]?\s*([\d.]+)",
            "conductivity": r"(?i)conductivity\s*\(?(?:µs/cm|us/cm)\)?\s*[:\-=]?\s*([\d.]+)",
        }

        extracted = {}
        for key, pattern in params.items():
```

```
            match = re.search(pattern, text)
            if match:
                try:
                    extracted[key] = float(match.group(1))
                except ValueError:
                    extracted[key] = None
            else:
                extracted[key] = None

        return jsonify({"success": True, "data": extracted})

    except Exception as e:
        return jsonify({"error": f"Error parsing PDF: {str(e)}"}), 500



@app.route("/dashboard")
@login_required
def dashboard():
    if session.get("role") != "user":
        return redirect(url_for("login"))

    if db is None or not db.is_connected():
        flash("Database connection error. Dashboard stats are unavailable.", "danger")
        stats = {"total": 0, "safe": 0, "unsafe": 0, "sources": {}, "history": []}
        return render_template("dashboard.html", stats=stats)

    try:
        user_id = session["user_id"]
        cursor.execute(
            "SELECT * FROM predictions WHERE user_id = %s ORDER BY created_at DESC",
            (user_id,),
        )
        predictions = cursor.fetchall()
```

```
    total = len(predictions)
    safe = sum(1 for p in predictions if p["prediction"] == "SAFE")
    unsafe = total - safe

    # Source analysis
    sources = {}
    history = []
    wqi_sum = 0
    monthly_data = {str(i): 0 for i in range(1, 13)}
    wqi_trend = []
    param_comparison = {'do': 0.0, 'ph': 0.0, 'bod': 0.0, 'nitrate': 0.0}

    for p in predictions:
        src = p["water_source"]
        sources[src] = sources.get(src, 0) + 1

        wqi_score, wqi_cat = calculate_wqi(p["dissolved_oxygen"], p["ph"], p["bod"], p["nitrate"], p["conductivity"])
        wqi_sum += wqi_score

        month = p["created_at"].month
        monthly_data[str(month)] += 1
        wqi_trend.append(wqi_score)

        param_comparison['do'] += p["dissolved_oxygen"]
        param_comparison['ph'] += p["ph"]
        param_comparison['bod'] += p["bod"]
        param_comparison['nitrate'] += p["nitrate"]

        history.append(
            {
                "id": p["id"],
                "source": src,
                "prediction_source": p.get("prediction_source", "Manual Entry"),
```

```
            "status": p["prediction"],
            "pollution_level": p["pollution_level"],
            "wqi": wqi_score,
            "wqi_category": wqi_cat,
            "timestamp": p["created_at"]
        }
        )

    avg_wqi = round(wqi_sum / total, 1) if total > 0 else 0
    wqi_trend = wqi_trend[:10][::-1]

    if total > 0:
        for k in param_comparison:
            param_comparison[k] = round(param_comparison[k] / total, 1)

    model_metrics = get_model_metrics()

    stats = {
        "total": total,
        "safe": safe,
        "unsafe": unsafe,
        "avg_wqi": avg_wqi,
        "sources": sources,
        "monthly": list(monthly_data.values()),
        "wqi_trend": wqi_trend,
        "param_comparison": param_comparison,
        "history": history[:10],
        "model_accuracy": model_metrics["accuracy"]
    }

    return render_template("dashboard.html", stats=stats)
except mysql.connector.Error as e:
    flash("A database error occurred loading the dashboard.", "danger")
    print(f"DB Error: {e}")
```

```
    stats = {"total": 0, "safe": 0, "unsafe": 0, "sources": {}, "history": []}
    return render_template("dashboard.html", stats=stats)


@app.route("/about")
def about():
    if session.get("logged_out"):
        return redirect(url_for("login"))
    return render_template("about.html")


@app.route("/contact", methods=["GET", "POST"])
def contact():
    if session.get("logged_out"):
        return redirect(url_for("login"))
    if request.method == "POST":
        name = request.form.get("name")
        email = request.form.get("email")
        subject = request.form.get("subject")
        message = request.form.get("message")

        if not (name and email and subject and message):
            flash("Please fill in all required fields before submitting.", "danger")
            return redirect(url_for("contact"))

        if not is_valid_name(name):
            flash("Name must contain letters and spaces only.", "danger")
            return redirect(url_for("contact"))

        if not is_valid_email(email):
            flash("Please enter a valid Gmail address ending with @gmail.com.", "danger")
            return redirect(url_for("contact"))

        if db is None or not db.is_connected():
```

```
            flash("Database connection error. Please try again later.", "danger")
            return redirect(url_for("contact"))

        try:
            user_id = session.get("user_id")
        for fb in feedbacks:
            if search:
                user_match = search in fb['user_name'].lower()
                insp_match = search in (fb['inspector_name'] or '').lower()
                comments_match = search in (fb['comments'] or '').lower()
                cats_match = search in (fb['categories'] or '').lower()
                recom_match = search in (fb['recommendation'] or '').lower()
                if not (user_match or insp_match or comments_match or cats_match or
recom_match):
                    continue
            filtered_feedbacks.append(fb)

        metrics = {
            'total': total_feedback,
            'avg_rating': avg_rating,
            'promoters': promoters,
            'passives': passives,
            'detractors': detractors,
            'nps_score': nps_score,
            'categories': cat_counts
        }

        return render_template('admin_feedbacks.html', feedbacks=filtered_feedbacks,
metrics=metrics, search=search, sort=sort)
    except Exception as e:
        flash(f"Error loading feedbacks: {e}", "danger")
        return redirect(url_for('admin_dashboard'))

@app.route('/feedback/<int:apt_id>', methods=['GET', 'POST'])
```

```
@login_required
def submit_feedback(apt_id):
    if session.get("role") != "user":
        flash("Unauthorized access.", "danger")
        return redirect(url_for('login'))

    try:
        # Fetch appointment details
        cursor.execute("""
            SELECT a.*, u.name as user_name, i.full_name as inspector_name
            FROM appointments a
            JOIN users u ON a.user_id = u.id
            LEFT JOIN inspectors i ON a.inspector_id = i.inspector_id
            WHERE a.appointment_id = %s AND a.user_id = %s
        """, (apt_id, session['user_id']))
        apt = cursor.fetchone()

        if not apt:
            flash("Appointment not found or unauthorized.", "danger")
            return redirect(url_for('history'))

        if apt['status'] != 'Completed':
            flash("You can only submit feedback for completed inspections.", "warning")
            return redirect(url_for('history'))

        # Check if feedback already exists to prevent duplicate submissions
        cursor.execute("SELECT feedback_id FROM feedbacks WHERE appointment_id = %s
AND user_id = %s", (apt_id, session['user_id']))
        existing = cursor.fetchone()
        if existing:
            flash("You have already submitted feedback for this inspection.", "info")
            return redirect(url_for('history'))

        if request.method == 'POST':
```

```
        rating = request.form.get("rating")
        if not rating:
            flash("Please select a rating.", "danger")
            return render_template('feedback.html', apt=apt)

        rating = int(rating)
        if rating < 1 or rating > 5:
            flash("Invalid rating selected.", "danger")
            return render_template('feedback.html', apt=apt)

        categories_list = request.form.getlist("categories")
        categories_str = ", ".join(categories_list) if categories_list else None
        comments = request.form.get("comments", "").strip() or None
        recommendation = request.form.get("recommendation", "Maybe")

        cursor.execute("""
            INSERT INTO feedbacks (appointment_id, user_id, rating, categories, comments, recommendation)
            VALUES (%s, %s, %s, %s, %s, %s)
        """, (apt_id, session['user_id'], rating, categories_str, comments, recommendation))
        db.commit()

        flash("Thank you for your feedback! Your response helps us improve AquaGuard AI.", "success")
        return redirect(url_for('history'))

    return render_template('feedback.html', apt=apt)
  except Exception as e:
    flash(f"Error handling feedback: {e}", "danger")
    return redirect(url_for('history'))

# --- UPI PAYMENT GATEWAY INTEGRATION ---
UPI_ID = os.getenv("UPI_ID", "aquaguard@upi")
UPI_MERCHANT_NAME = os.getenv("UPI_MERCHANT_NAME", "AquaGuard AI")
```

```
@app.route('/payment/<int:report_id>')
@login_required
def payment(report_id):
    if session.get("role") != "user":
        flash("Please log in to view this page.", "danger")
        return redirect(url_for('login'))

    try:
        # Verify ownership and completion
        cursor.execute("""
            SELECT r.report_id, a.appointment_id, a.status
            FROM inspection_reports r
            JOIN appointments a ON r.appointment_id = a.appointment_id
            WHERE r.report_id=%s AND a.user_id=%s
        """, (report_id, session['user_id']))
        report = cursor.fetchone()
        if not report:
            flash("Report not found or unauthorized.", "danger")
            return redirect(url_for('history'))
        if report['status'] != 'Completed':
            flash("Report is locked because inspection is not fully completed.", "danger")
            return redirect(url_for('history'))

        # Check if already paid
        cursor.execute("SELECT transaction_id FROM payments WHERE report_id = %s AND
status = 'Paid'", (report_id,))
        paid = cursor.fetchone()
        if paid:
            return redirect(url_for('payment_success', report_id=report_id))

        return render_template(
            'payment.html',
            report_id=report_id,
```

```
            amount=50,
            upi_id=UPI_ID,
            upi_merchant_name=UPI_MERCHANT_NAME
        )
    except Exception as e:
        flash(f"Error: {e}", "danger")
        return redirect(url_for('history'))

@app.route('/process_payment/<int:report_id>', methods=['POST'])
@login_required
@csrf.exempt
def process_payment(report_id):
    if session.get("role") != "user":
        return jsonify({'error': 'Unauthorized'}), 401

    data = request.get_json(silent=True) or request.form
    name = data.get('name', '').strip()
    email = data.get('email', '').strip()
    phone = data.get('phone', '').strip()
    payment_method = data.get('payment_method', 'Google Pay').strip()

    # Server-side Validations
    if not name or not re.match(r"^[a-zA-Z\s]+$", name) or len(name) < 3:
        return jsonify({'error': 'Invalid name. Must contain only letters and spaces, min 3 chars.'}),
400

    if not email or not re.match(r"[^@]+@[^@]+\.[^@]+", email):
        return jsonify({'error': 'Invalid email format.'}), 400

    if not phone or not re.match(r"^\d{10}$", phone):
        return jsonify({'error': 'Phone number must be exactly 10 digits.'}), 400

    try:
        # Verify ownership and completion
```

```
    cursor.execute("""
        SELECT r.report_id, a.appointment_id, a.status
        FROM inspection_reports r
        JOIN appointments a ON r.appointment_id = a.appointment_id
        WHERE r.report_id=%s AND a.user_id=%s
    """, (report_id, session['user_id']))
    report = cursor.fetchone()
    if not report:
        return jsonify({'error': 'Report not found or unauthorized.'}), 404
    if report['status'] != 'Completed':
        return jsonify({'error': 'Report is locked because inspection is not fully completed.'}), 400

    # Check duplicate
    cursor.execute("SELECT id FROM payments WHERE report_id = %s AND status = 'Paid'", (report_id,))
    if cursor.fetchone():
        return jsonify({'redirect_url': url_for('payment_success', report_id=report_id)}), 200

    amount_in_rupees = 50
    timestamp_sec = int(datetime.now().timestamp())
    txn_ref = f"UPI_REP_{report_id}_{timestamp_sec}"

    # Insert or update pending payment record
    cursor.execute("SELECT id FROM payments WHERE report_id = %s AND status = 'Pending'", (report_id,))
    pending_payment = cursor.fetchone()
    if pending_payment:
        cursor.execute("""
            UPDATE payments
            SET transaction_id = %s, amount = %s, payment_method = %s, payment_date = NOW(), appointment_id = %s
            WHERE id = %s
        """, (txn_ref, amount_in_rupees, payment_method, report['appointment_id'],
```

```
pending_payment['id']))
        else:
            cursor.execute("""
                INSERT INTO payments (user_id, amount, status, appointment_id, report_id, payment_method, transaction_id, payment_date)
                VALUES (%s, %s, %s, %s, %s, %s, %s, NOW())
            """, (session['user_id'], amount_in_rupees, 'Pending', report['appointment_id'], report_id, payment_method, txn_ref))
        db.commit()

        from urllib.parse import quote
        tn = quote(f"AquaGuard Report Access #{report_id}")
        pn = quote(UPI_MERCHANT_NAME)
        upi_uri = f"upi://pay?pa={UPI_ID}&pn={pn}&am={amount_in_rupees:.2f}&cu=INR&tn={tn}&tr={txn_ref}"
        gpay_uri = f"tez://upi/pay?pa={UPI_ID}&pn={pn}&am={amount_in_rupees:.2f}&cu=INR&tn={tn}&tr={txn_ref}"
        phonepe_uri = f"phonepe://pay?pa={UPI_ID}&pn={pn}&am={amount_in_rupees:.2f}&cu=INR&tn={tn}&tr={txn_ref}"

        return jsonify({
            'success': True,
            'txn_ref': txn_ref,
            'amount': amount_in_rupees,
            'currency': 'INR',
            'upi_id': UPI_ID,
            'upi_merchant_name': UPI_MERCHANT_NAME,
            'payment_method': payment_method,
            'upi_uri': upi_uri,
            'gpay_uri': gpay_uri,
            'phonepe_uri': phonepe_uri,
```

```
            'report_id': report_id
        }), 200
    except Exception as e:
        print("UPI order creation exception:", e)
        return jsonify({'error': f'Payment failed to initialize: {str(e)}'}), 500

@app.route('/verify_upi_payment', methods=['POST'])
@login_required
@csrf.exempt
def verify_upi_payment():
    if session.get("role") != "user":
        return jsonify({'error': 'Unauthorized'}), 401

    try:
        data = request.get_json(silent=True) or request.form
        report_id = data.get('report_id')
        transaction_ref = data.get('transaction_ref') or data.get('txn_ref')
        utr_number = (data.get('utr_number') or data.get('utr') or '').strip()
        payment_method = data.get('payment_method', 'UPI').strip()
        action = data.get('action', 'verify')

        if not report_id:
            return jsonify({'status': 'failed', 'message': 'Missing report ID.'}), 400

        report_id = int(report_id)

        if action == 'cancel':
            cursor.execute("SELECT id FROM payments WHERE report_id = %s AND status =
'Pending'", (report_id,))
            p_row = cursor.fetchone()
            if p_row:
                cursor.execute("UPDATE payments SET status = 'Failed' WHERE id = %s",
(p_row['id'],))
                db.commit()
```

```
        return jsonify({'status': 'cancelled', 'message': 'Payment cancelled by user.'}), 200

    if not utr_number or len(utr_number) < 6:
        return jsonify({
            'status': 'failed',
            'message': 'Please enter a valid 12-digit UPI UTR / Reference Number from your payment app.'
        }), 400

    cursor.execute("""
        SELECT r.report_id, a.appointment_id
        FROM inspection_reports r
        JOIN appointments a ON r.appointment_id = a.appointment_id
        WHERE r.report_id=%s AND a.user_id=%s
    """, (report_id, session['user_id']))
    report_row = cursor.fetchone()
    if not report_row:
        return jsonify({'status': 'failed', 'message': 'Report not found or unauthorized.'}), 404

    cursor.execute("SELECT id FROM payments WHERE (transaction_id = %s OR report_id = %s) AND status != 'Paid'", (transaction_ref, report_id))
    payment_row = cursor.fetchone()

    final_txn_id = utr_number if utr_number else (transaction_ref or f"UPI_{report_id}_{int(datetime.now().timestamp())}")

    if payment_row:
        cursor.execute("""
            UPDATE payments
            SET status = 'Paid', transaction_id = %s, payment_method = %s, payment_date = NOW()
            WHERE id = %s
        """, (final_txn_id, payment_method, payment_row['id']))
    else:
```

```
            cursor.execute("""
                INSERT INTO payments (user_id, amount, status, appointment_id, report_id, payment_method, transaction_id, payment_date)
                VALUES (%s, %s, %s, (SELECT appointment_id FROM inspection_reports WHERE report_id = %s), %s, %s, %s, NOW())
            """, (session['user_id'], 50, 'Paid', report_id, report_id, payment_method, final_txn_id))
        db.commit()

        flash("Payment completed and verified successfully!", "success")
        return jsonify({
            'status': 'success',
            'redirect_url': url_for('payment_success', report_id=report_id)
        }), 200
    except Exception as e:
        print("UPI payment verification error:", e)
        return jsonify({'status': 'failed', 'message': f'Verification error: {str(e)}'}), 500

@app.route('/payment_success/<int:report_id>')
@login_required
def payment_success(report_id):
    if session.get("role") != "user":
        return redirect(url_for('login'))

    try:
        cursor.execute("""
            SELECT amount, transaction_id, payment_method, payment_date
            FROM payments
            WHERE report_id = %s AND user_id = %s AND status = 'Paid'
            ORDER BY id DESC LIMIT 1
        """, (report_id, session['user_id']))
        payment = cursor.fetchone()
        if not payment:
            flash("No successful payment found for this report.", "danger")
            return redirect(url_for('history'))
```

```
        return render_template('payment_success.html', report_id=report_id, payment=payment)
    except Exception as e:
        flash(f"Error: {e}", "danger")
        return redirect(url_for('history'))

@app.route('/download_inspection_report/<int:report_id>')
@login_required
def download_inspection_report(report_id):
    role = session.get("role")
    if not role: return redirect(url_for('login'))
    try:
        report = None
        if role == 'user':
            cursor.execute("""
                SELECT r.report_file, a.status
                FROM inspection_reports r
                JOIN appointments a ON r.appointment_id = a.appointment_id
                WHERE r.report_id=%s AND a.user_id=%s
                AND (SELECT COUNT(*) FROM payments p WHERE p.report_id = r.report_id
AND p.status = 'Paid') > 0
            """, (report_id, session['user_id']))
            report = cursor.fetchone()
            if not report:
                flash("Report not found, incomplete, or unpaid.", "danger")
                return redirect(url_for('history'))
            if report['status'] != 'Completed':
                flash("Report is locked because inspection is not fully completed.", "danger")
                return redirect(url_for('history'))
        elif role == 'inspector':
            cursor.execute("SELECT report_file FROM inspection_reports WHERE report_id=%s
AND inspector_id=%s", (report_id, session['user_id']))
            report = cursor.fetchone()
        elif role == 'admin':
```

```
            flash("Access denied. Admin does not have permission to view or download detailed inspection reports.", "danger")
            return redirect(url_for('admin_appointments'))

        if not report:
            flash("Report not found or unauthorized.", "danger")
            if role == 'user': return redirect(url_for('history'))
            elif role == 'inspector': return redirect(url_for('inspector_dashboard'))
            else: return redirect(url_for('admin_appointments'))

        path = os.path.join(app.root_path, report['report_file'].lstrip('/'))
        from flask import send_file
        return send_file(path, as_attachment=True)
    except Exception as e:
        flash(f"Error downloading: {e}", "danger")
        return redirect(url_for('index'))


# --- DYNAMIC WATER QUALITY REPORT GENERATOR (REPORTLAB) ---

from reportlab.pdfgen import canvas
from reportlab.lib import colors
from reportlab.graphics.shapes import Drawing, Rect, String, Wedge, Circle, Line

class NumberedCanvas(canvas.Canvas):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.pages = []

    def showPage(self):
        self.pages.append(dict(self.__dict__))
        self._startPage() # type: ignore

    def save(self):
```

```
        page_count = len(self.pages)
        for page in self.pages:
            self.__dict__.update(page)
            self.draw_footer(page_count)
            super().showPage()
        super().save()

    def draw_footer(self, page_count):
        self.saveState()
        self.setFont("Helvetica", 9)
        self.setFillColor(colors.HexColor("#64748b"))
        self.drawString(54, 30, "Generated by AquaGuard AI – AI-Based Water Quality Monitoring System")
        self.drawRightString(612 - 54, 30, f"Page {self._pageNumber} of {page_count}") # type: ignore
        self.setStrokeColor(colors.HexColor("#cbd5e1"))
        self.setLineWidth(0.5)
        self.line(54, 42, 612 - 54, 42)
        self.restoreState()

def check_parameter_status(name, value):
    if value is None:
        return 'Normal'
    try:
        val = float(value)
    except ValueError:
        return 'Normal'
    story.append(summary_table)
    story.append(Spacer(1, 12))

    p_wqi, p_wqi_cat = calculate_wqi(
        prediction['dissolved_oxygen'],
        prediction['ph'],
        prediction['bod'],
```

```
        prediction['nitrate'],
        prediction['conductivity']
    )

    is_safe = prediction['prediction'] == 'SAFE'
    bg_color = colors.HexColor('#d1fae5') if is_safe else colors.HexColor('#fee2e2')
    border_color = colors.HexColor('#34d399') if is_safe else colors.HexColor('#fca5a5')
    text_color = '#065f46' if is_safe else '#991b1b'
    status_label = "SAFE" if is_safe else "UNSAFE"

    status_text = f"<font size=14 color='{text_color}'><b>AI ASSESSMENT: {status_label}</b></font>"
    explanation = (
        f"The AI water assessment model predicts that the water quality for this sample is <b>{status_label}</b>. "
        f"The calculated Water Quality Index (WQI) is <b>{p_wqi}</b>, which classifies the sample under the <b>'{p_wqi_cat}'</b> category. "
    )
    if is_safe:
        explanation += "Water with this score is generally suitable for drinking, irrigation, and household use without immediate restriction."
    else:
        explanation += "WARNING: Water contains high levels of target parameters. Boiling or multi-stage filtration purification is strongly recommended."

    result_para = Paragraph(f"{status_text}<br/>{explanation}", alert_style)
        cond = request.form.get("conductivity")
        nitrate = request.form.get("nitrate")
        faecal_strep = request.form.get("faecal_streptococci")
        faecal_coli = request.form.get("faecal_coliform")
        total_coli = request.form.get("total_coliform")

        insp_date = request.form.get("inspection_date")
        insp_time = request.form.get("inspection_time")
```

```
    water_src = request.form.get("water_source")
    water_app = request.form.get("water_appearance")
    water_col = request.form.get("water_colour")
    water_odour = request.form.get("water_odour")
    vis_cont = request.form.get("visible_contamination")
    near_poll = request.form.get("nearby_pollution")
    drain_leak = request.form.get("drainage_leakage")
    samp_coll = request.form.get("sample_collected")
    rem = request.form.get("inspector_remarks")
    assess = request.form.get("inspector_assessment")
    pdf_path = request.form.get("pdf_path") or None

    # New fields
    sample_id = request.form.get("sample_id")
    sample_collection_time = request.form.get("sample_collection_time") or None
    sample_type = request.form.get("sample_type")
    sample_count = request.form.get("sample_count") or None
    container_type = request.form.get("container_type")
    preservation_method = request.form.get("preservation_method")
    sample_sent_to_lab = request.form.get("sample_sent_to_lab")
    laboratory_name = request.form.get("laboratory_name")
    laboratory_ref = request.form.get("laboratory_ref")
    inspection_summary = request.form.get("inspection_summary")
    recommended_action = request.form.get("recommended_action")

    if appointment_id:
        cursor.execute("SELECT inspection_type, status, user_id FROM appointments
WHERE appointment_id = %s", (appointment_id,))
        apt_chk = cursor.fetchone()
        if apt_chk:
            if apt_chk['inspection_type'] == 'Online Inspection':
                flash("Field Inspection is not available for this Online-Only inspection.",
"warning")
                return redirect(url_for('inspector_dashboard'))
```

```
            if apt_chk.get('user_id'):
                user_id = apt_chk['user_id']

        # Validation of required fields
        if not all([insp_date, insp_time, water_src, water_app, water_col, water_odour, vis_cont,
near_poll, drain_leak, samp_coll, assess]):
            flash("Please complete all required fields before submitting the inspection.", "danger")
            return redirect(url_for('inspector_field_inspection', appointment_id=appointment_id))

        # Range and numerical type validation
        try:
            if temperature and str(temperature).strip():
                t_val = float(temperature)
                if t_val < -50 or t_val > 100:
                    flash("Temperature is out of valid scientific range (-50 to 100).", "danger")
                    return                                    redirect(url_for('inspector_field_inspection',
appointment_id=appointment_id))
            if ph and str(ph).strip():
                ph_val = float(ph)
                if not (0 <= ph_val <= 14):
                    flash("pH must be between 0 and 14.", "danger")
                    return                                    redirect(url_for('inspector_field_inspection',
appointment_id=appointment_id))
            for val, name in [
                (do, "Dissolved Oxygen"),
                (bod, "BOD"),
                (cond, "Conductivity"),
                (nitrate, "Nitrate"),
                (faecal_strep, "Faecal Streptococci"),
                (faecal_coli, "Faecal Coliform"),
                (total_coli, "Total Coliform")
            ]:
                if val and str(val).strip():
                    v_val = float(val)
```

```
            if v_val < 0:
                flash(f"{name} cannot be negative.", "danger")
                return redirect(url_for('inspector_field_inspection', appointment_id=appointment_id))
        except ValueError:
            flash("Please enter a valid numeric value.", "danger")
            return redirect(url_for('inspector_field_inspection', appointment_id=appointment_id))

        try:
            cursor.execute('''
                INSERT INTO field_inspections (
                    inspector_id, user_id, appointment_id, temperature, ph, dissolved_oxygen, bod,
                    conductivity, nitrate, faecal_streptococci, faecal_coliform, total_coliform,
                    inspection_date, inspection_time, water_source, water_appearance, water_colour,
                    water_odour, visible_contamination, nearby_pollution, drainage_leakage, sample_collected,
                    inspector_remarks, inspector_assessment, pdf_path,
                    sample_id, sample_collection_time, sample_type, sample_count, container_type,
                    preservation_method, sample_sent_to_lab, laboratory_name, laboratory_ref,
                    inspection_summary, recommended_action
            ''', (session["user_id"], user_id, appointment_id, temperature, ph, do, bod, cond, nitrate, faecal_strep, faecal_coli, total_coli, insp_date, insp_time, water_src, water_app, water_col, water_odour, vis_cont, near_poll, drain_leak, samp_coll, rem, assess, pdf_path,
                  sample_id, sample_collection_time, sample_type, sample_count, container_type, preservation_method, sample_sent_to_lab, laboratory_name, laboratory_ref, inspection_summary, recommended_action))
            inspection_id = cursor.lastrowid

            # Save Photos
            upload_folder = os.path.join(app.root_path, 'static', 'uploads', 'inspections')
            os.makedirs(upload_folder, exist_ok=True)
            for photo_key in ['photo_sample', 'photo_source', 'photo_surrounding', 'photo_pollution', 'photo_equipment', 'photo_additional']:
                if photo_key in request.files:
```

```
            files = request.files.getlist(photo_key)
            for file in files:
                if file and file.filename != '':
                    filename = secure_filename(f"{inspection_id}_{photo_key}_{file.filename}")
                    file_path = os.path.join(upload_folder, filename)
                    file.save(file_path)
                    desc = request.form.get(f"{photo_key}_desc", "")
                    cursor.execute("INSERT INTO field_inspection_photos (inspection_id, photo_type, file_path, description) VALUES (%s, %s, %s, %s)", (inspection_id, photo_key, f"/static/uploads/inspections/{filename}", desc))

        # Save Techniques
        techniques_json = request.form.get("techniques_data")
        if techniques_json:
            techniques = json.loads(techniques_json)
            for tech in techniques:
                cursor.execute("""
                    INSERT INTO field_inspection_techniques (
                        inspection_id, technique, notes, equipment_name, equipment_model,
                        instrument_id, calibration_status, calibration_date, measurement_unit, remarks

                """, (inspection_id, tech.get('technique'), tech.get('notes'), tech.get('equipment_name'), tech.get('equipment_model'), tech.get('instrument_id'), tech.get('calibration_status'), tech.get('calibration_date') or None, tech.get('measurement_unit'), tech.get('remarks')))

        # Save Measurements
        measurements_json = request.form.get("measurements_data")
        if measurements_json:
            measurements = json.loads(measurements_json)
            for meas in measurements:
                cursor.execute("""
```

```
                INSERT INTO field_inspection_measurements (
                    inspection_id, parameter, value, unit, technique, instrument, measurement_time, remarks
                ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (inspection_id, meas.get('parameter'), meas.get('value') or None, meas.get('unit'), meas.get('technique'), meas.get('instrument'), meas.get('time') or None, meas.get('remarks')))

        if appointment_id:
            cursor.execute("UPDATE appointments SET field_status='Completed' WHERE appointment_id=%s", (appointment_id,))
            db.commit()
            finalize_appointment_report(appointment_id, session['user_id'])
            flash("Field Inspection saved and sub-status updated successfully.", "success")
            return redirect(url_for('inspector_dashboard'))
        else:
            db.commit()
            flash("Field Inspection saved successfully.", "success")
            return redirect(url_for('inspector_history'))
    except Exception as e:
        flash(f"Error saving inspection: {e}", "danger")
        db.rollback()

# GET request
appointment_id = request.args.get("appointment_id")
selected_user_id = None
appointment_details = None
if appointment_id:
    cursor.execute("""
        SELECT a.*, u.name as user_name, u.email, u.phone_number
        FROM appointments a
        LEFT JOIN users u ON a.user_id = u.id
        WHERE a.appointment_id = %s
    """, (appointment_id,))
```

```
        apt_row = cursor.fetchone()
        if apt_row:
            if apt_row['status'] in ('Pending Inspector Approval', 'Pending') or apt_row.get('assignment_status') == 'Pending Inspector Approval':
                flash("You must approve the inspection assignment before starting field inspection.", "danger")
                return redirect(url_for('inspector_dashboard'))
            if apt_row['inspection_type'] == 'Online Inspection':
                flash("Field Inspection is not available for this Online-Only appointment.", "warning")
                return redirect(url_for('inspector_dashboard'))
            selected_user_id = apt_row['user_id']
            appointment_details = make_json_serializable(apt_row)

    cursor.execute("SELECT id, name, email, phone_number FROM users WHERE role='user'")
    users = cursor.fetchall()

    now = datetime.now()
    return render_template('field_inspection.html', users=users, today=now.strftime("%Y-%m-%d"), time_now=now.strftime("%H:%M"), appointment_id=appointment_id, selected_user_id=selected_user_id, appointment_details=appointment_details)

@app.route('/inspector/profile', methods=['GET', 'POST'])
@login_required
def inspector_profile():
    if session.get("role") != "inspector": return redirect(url_for("login"))
    insp_id = session.get("user_id")

    if request.method == 'POST':
        action = request.form.get('action')
        if action == 'update_profile':
            fn = request.form.get('full_name', '').strip()
            ph = request.form.get('phone', '').strip()
```

```
        if not fn or not ph:
            flash("Please fill in all required fields before submitting.", "danger")
        elif not is_valid_name(fn):
            flash("Name must contain letters and spaces only.", "danger")
        elif not is_valid_phone(ph):
            flash("Phone number must contain exactly 10 digits.", "danger")
        else:
            try:
                cursor.execute("UPDATE inspectors SET full_name=%s, phone=%s WHERE inspector_id=%s", (fn, ph, insp_id))
                db.commit()
                session['name'] = fn
                flash("Profile updated successfully.", "success")
            except Exception as e:
                flash("Error updating profile.", "danger")
    elif action == 'change_password':
        curr = request.form.get('current_password')
        new_p = request.form.get('new_password')
        conf = request.form.get('confirm_password')

        if not new_p or not conf or not curr:
            flash("Please fill in all required fields before submitting.", "danger")
        elif not is_valid_password(new_p):
            flash("Password must contain at least one letter and one number.", "danger")
        elif new_p != conf:
            flash("Password and Confirm Password do not match.", "danger")
        else:
            cursor.execute("SELECT password FROM inspectors WHERE inspector_id=%s", (insp_id,))
            res = cursor.fetchone()
            if not res or not check_password_hash(res['password'], curr):
                flash("Incorrect current password.", "danger")
            else:
                cursor.execute("UPDATE inspectors SET password=%s WHERE
```

```
inspector_id=%s", (generate_password_hash(new_p), insp_id))
                db.commit()
                flash("Password changed successfully.", "success")

    cursor.execute("SELECT * FROM inspectors WHERE inspector_id=%s", (insp_id,))
    inspector = cursor.fetchone()

    cursor.execute("SELECT COUNT(*) as c FROM appointments WHERE inspector_id=%s",
(insp_id,))
    assigned = cursor.fetchone()['c']
    cursor.execute("SELECT COUNT(*) as c FROM appointments WHERE inspector_id=%s
AND status='Completed'", (insp_id,))
    completed = cursor.fetchone()['c']

    return render_template('inspector_profile.html', inspector=inspector, stats={'assigned':
assigned, 'completed': completed})

@app.route('/inspector/history')
@login_required
def inspector_history():
    if session.get("role") != "inspector": return redirect(url_for("login"))

    cursor.execute('''
        SELECT f.*, u.name as user_name, p.prediction as ai_prediction
        FROM field_inspections f
        LEFT JOIN users u ON f.user_id = u.id
        LEFT JOIN appointments a ON f.appointment_id = a.appointment_id
        LEFT JOIN predictions p ON a.prediction_id = p.id
        WHERE f.inspector_id = %s
        ORDER BY f.created_at DESC
    ''', (session["user_id"],))
    inspections = cursor.fetchall()

    return render_template('inspector_history.html', inspections=inspections)
```

```
if __name__ == "__main__":
    app.run(debug=True)
```